Advanced big data analytics

Assignment 2: Mobile money investigations

10/27/26

Member	Student ID	Contribution
BUYANGE B. Aline	101367	Everyone contributed equally to this work. We worked together; we had to make calls at least 2 times, and we generated the report together. No one was left behind.
MUGABO MUTSINZI CHRISTIAN	101226	
UWIZEYE CHRISTELLE	101465	

Adventist University of Central Africa
P.O. Box 2461 Kigali, Rwanda | www.auca.ac.rw | info@auca.ac.rw

Contents

Summary 3
1 Prepare and capture the data 4
1.1 Environment 4
1.2 Capture the generator streams 7
2 PostgreSQL model 9
2.1 Schema 9
2.2 Load, keys and indexes 12
3 Neo4j model 15
3.1 Graph schema 15
3.2 Bulk import 15
3.3 Trace account 998 in both models 16
4 Routine business questions 18
4.1 Q1 Account activity 18
4.2 Q2 Daily activity 19
4.3 Q3 Shared devices 21
5 Network and investigation questions 23
5.1 Q4 Ownership and device network 23
5.2 Q5 Rapid onward payments 25
5.3 Q6 Circular payments 26
5.4 Q7 Shortest person-only path 27
6 Correctness and timing 29
6.1 One comparison program 29
6.2 Timing a fixed selection 30
6.3 One SQL improvement (Q7) 35
7 Concurrent requests 38
7.1 Load test 38
7.2 Scaling 39
8 Decisions 41
8.1 Prioritise investigation 41
8.2 Recommend a model 42
8.3 Individual judgement 43

Summary

We generated one month of synthetic mobile money data (20,993,500 rows, 20,003,300 of them transfers), loaded it into PostgreSQL as five tables and into Neo4j as a graph, and answered seven investigation questions in both. Every one of our 61 comparison cases gave the same answer in both engines, and two answers were also checked straight from the CSV files with plain Python.

Main findings. PostgreSQL was faster for simple indexed lookups (0.1 to 0.9 ms against 2.4 to 4.7 ms) and much faster for the monthly report Q2 (1.22 s against 23.4 s). Neo4j was much faster for shortest paths with our first SQL (13.8 s against 7.5 ms), but most of that gap came from the search method: after we rewrote the SQL to search from both ends it took 26 to 28 ms. Under 16 concurrent clients Neo4j still served about 9 times (Q4) and 23 times (Q7) more requests per second. In the data, accounts A, B and C show quick onward payment chains (and A and C close cycles), while D, E and F have the same kind of device links but slow, flat payments. We recommend PostgreSQL for the record and reporting, and Neo4j as a refreshed copy for multi-step investigation.

Milestone	Status / key evidence
1 Capture	5 streams, 20,993,500 rows, all counts match; transfer audit sum 231,482,327,870 RWF
2 PostgreSQL	Load 11.03 s, keys+indexes 44.86 s, 15 constraints validated, 3.72 GB
3 Neo4j	Import 22.72 s, constraints 17.89 s, 704,600 nodes, 20,536,900 relationships, 2.06 GB
4-5 Q1-Q7	All six review sets answered in both engines
6.1 Comparison	61/61 match, all Q7 paths valid, 2 CSV checks agree
6.2-6.3 Timing	21 cases x 2 engines, no timeouts; Q7 SQL 495-551x faster after rewrite
7 Load test	48 tests, 0 errors, 0 timeouts
8 Decisions	Review order C, A, B; PostgreSQL + Neo4j split

1 Prepare and capture the data

1.1 Environment

We worked on one laptop for the whole project so that timings are comparable. The generator file was checked with SHA-256 at the start and the checksum was saved in docs/generator_checksum.txt (2ea0e59d…e9455a).

Table 3: Environment used for every measurement

Item	Value
Machine	Apple M1 (4 performance + 4 efficiency cores), 16 GB RAM, macOS 26.6.2
Storage	Project and CSVs on external disk /Volumes/Winner (734 GiB free)
PostgreSQL	17.2 (EDB, x86_64 under Rosetta 2), database mobile_money, timezone Africa/Kigali
Neo4j	2026.10.0 Community (Homebrew), Cypher 5 and 25; heap 4 GiB initial and max, page cache 3 GiB
Java	OpenJDK 21.0.13 (set in ~/.zshrc; Java 17 was the default before)
Python	3.14.4 virtual environment with psycopg 3, neo4j driver, numpy, pandas, matplotlib, psutil

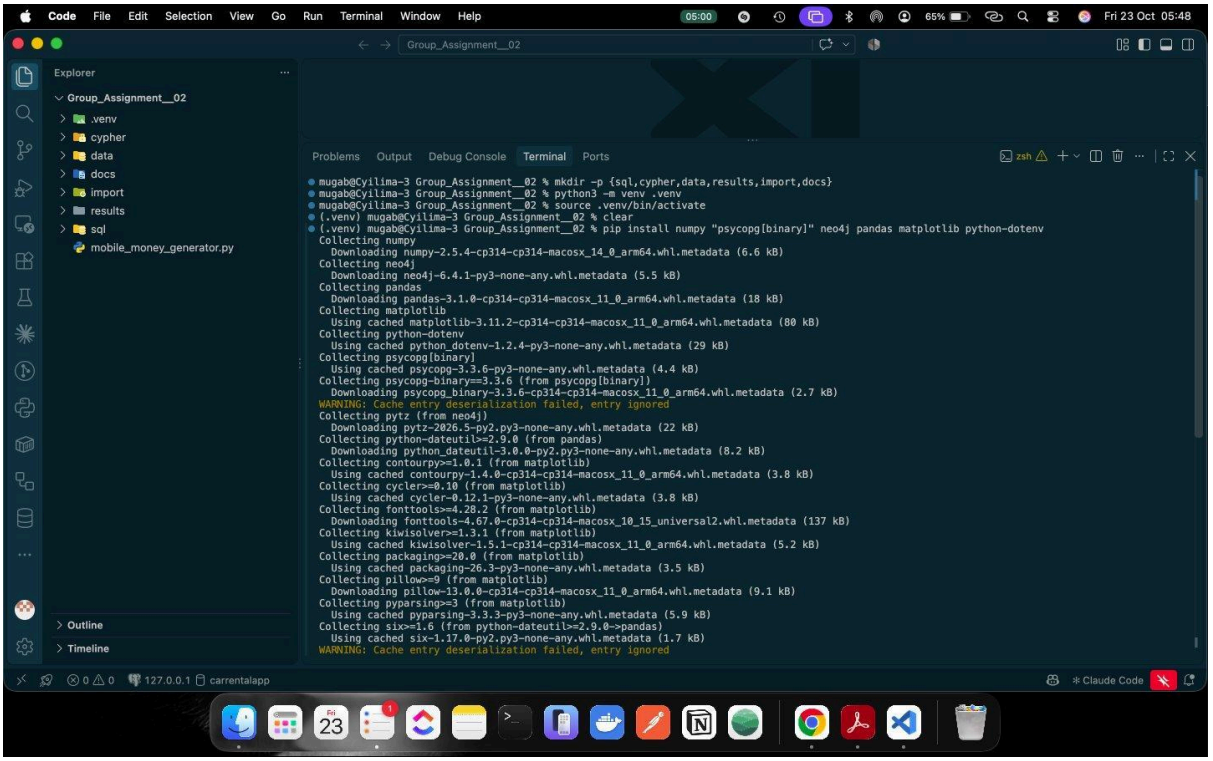


Figure 1: Project folders and Python environment created

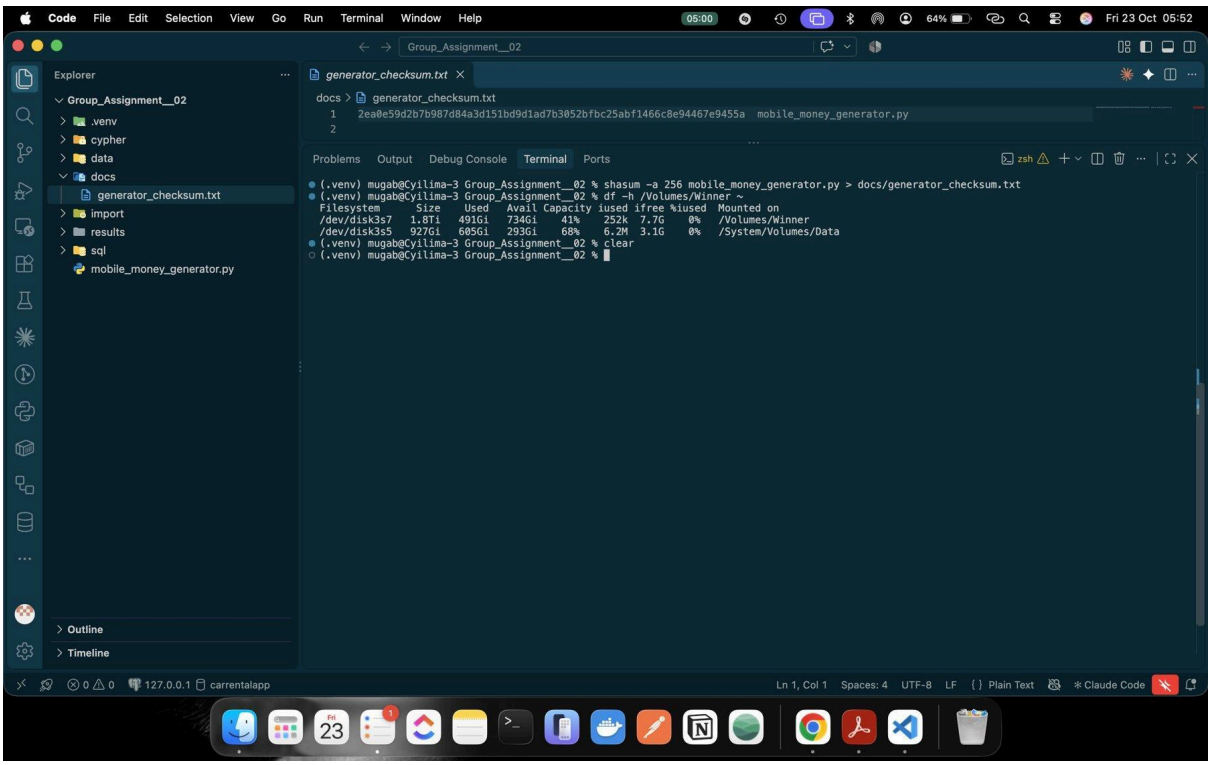


Figure 2: Generator checksum saved and free disk space checked

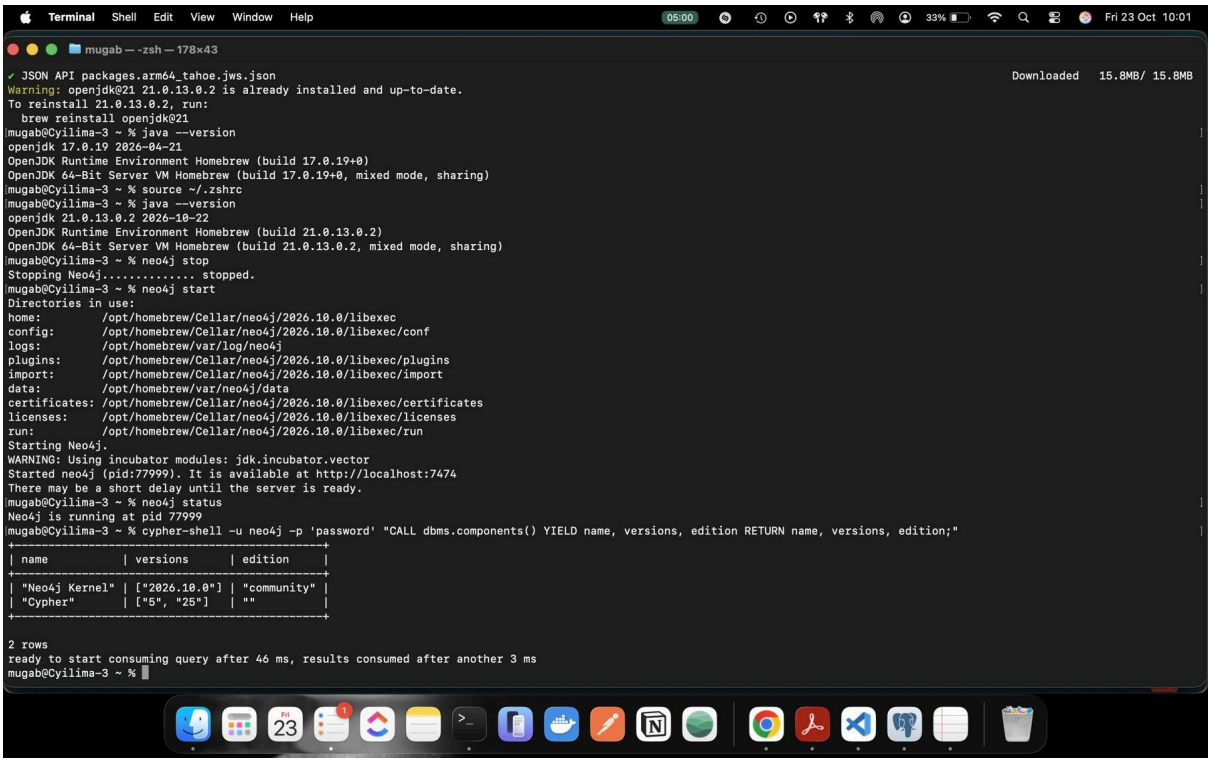


Figure 3: Java 21 selected for Neo4j

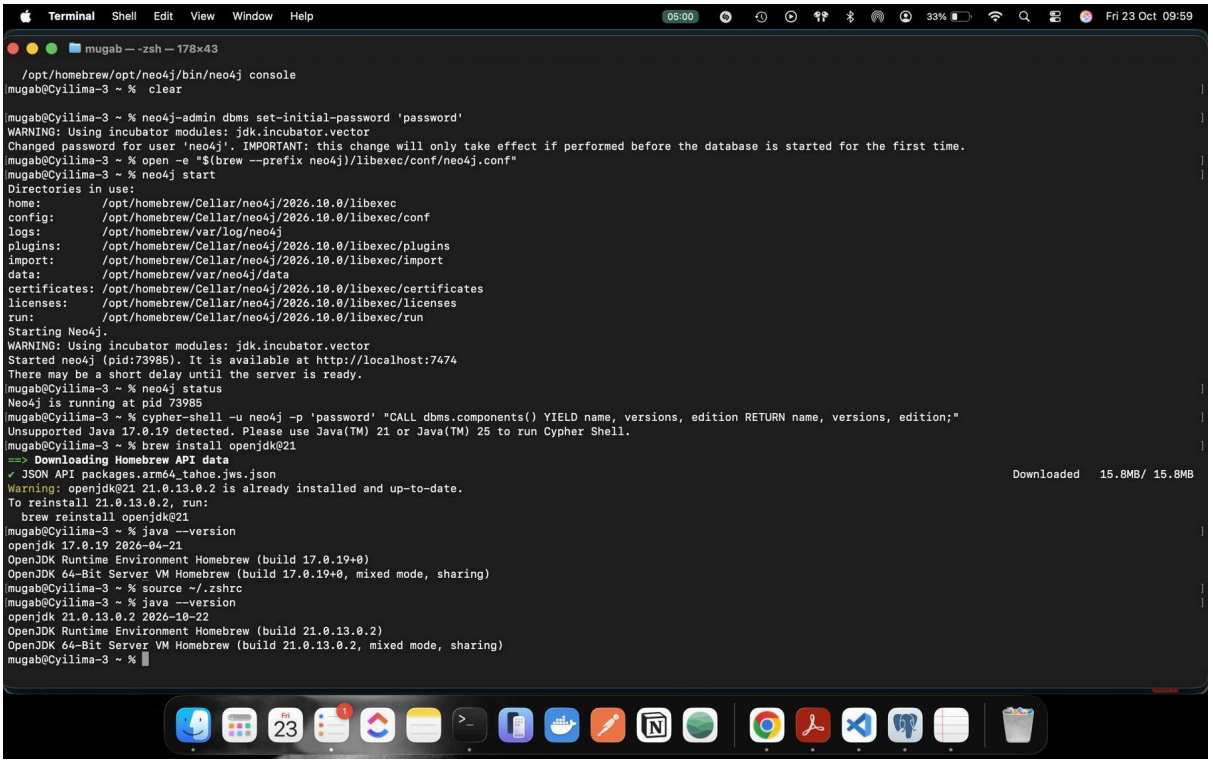


Figure 4: Neo4j initial password, configuration and start

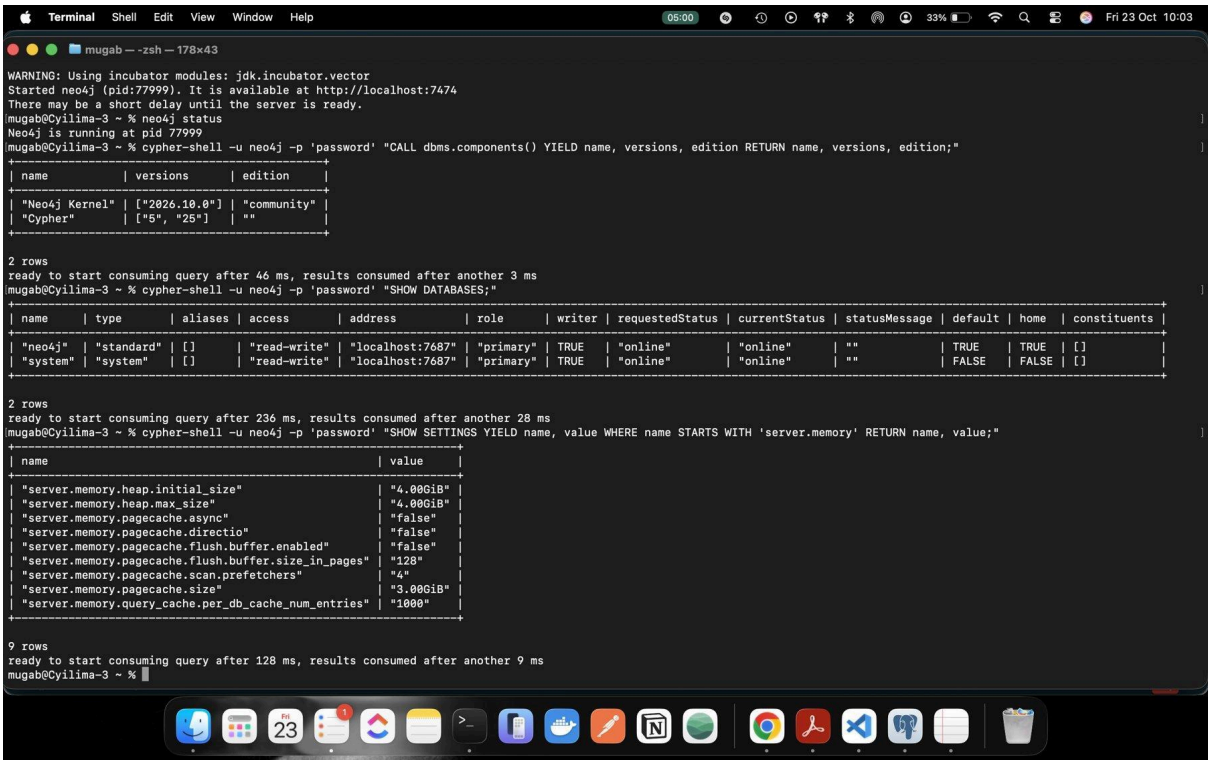


Figure 5: Neo4j running, kernel 2026.10.0 Community

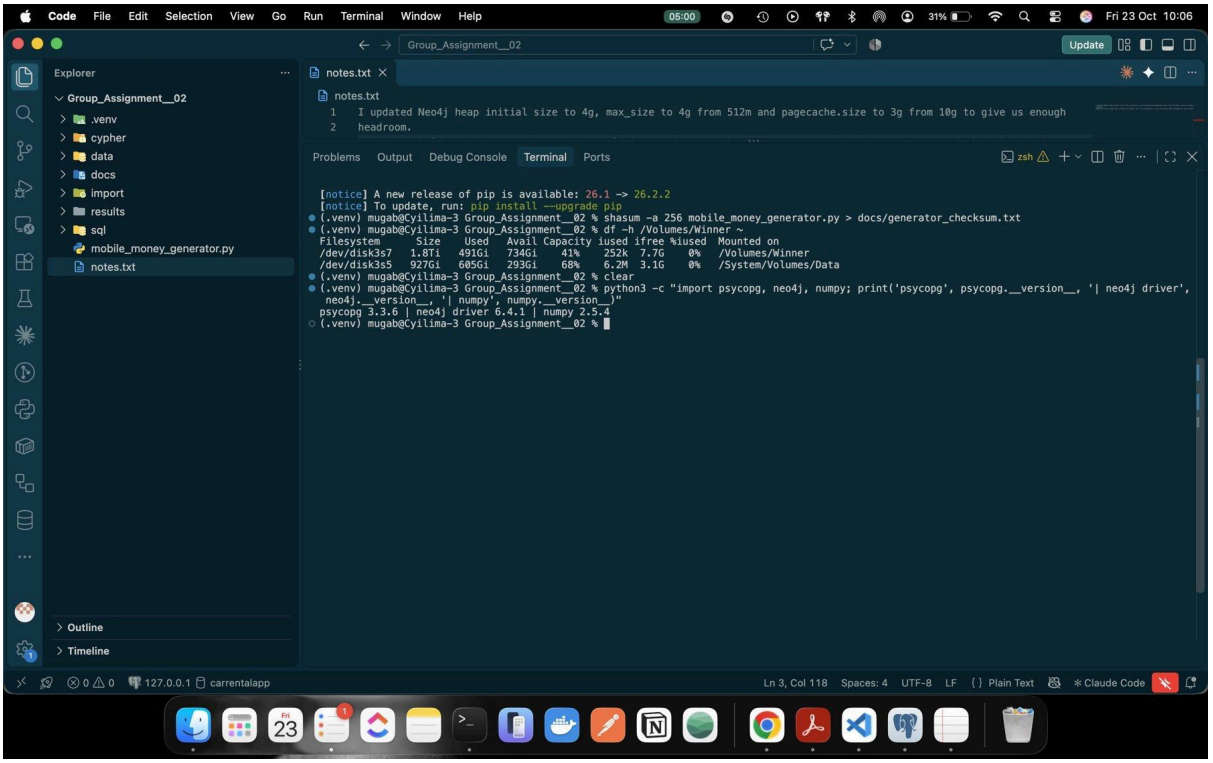


Figure 6: Our note on the Neo4j memory settings (heap 4 GiB, page cache 3 GiB)

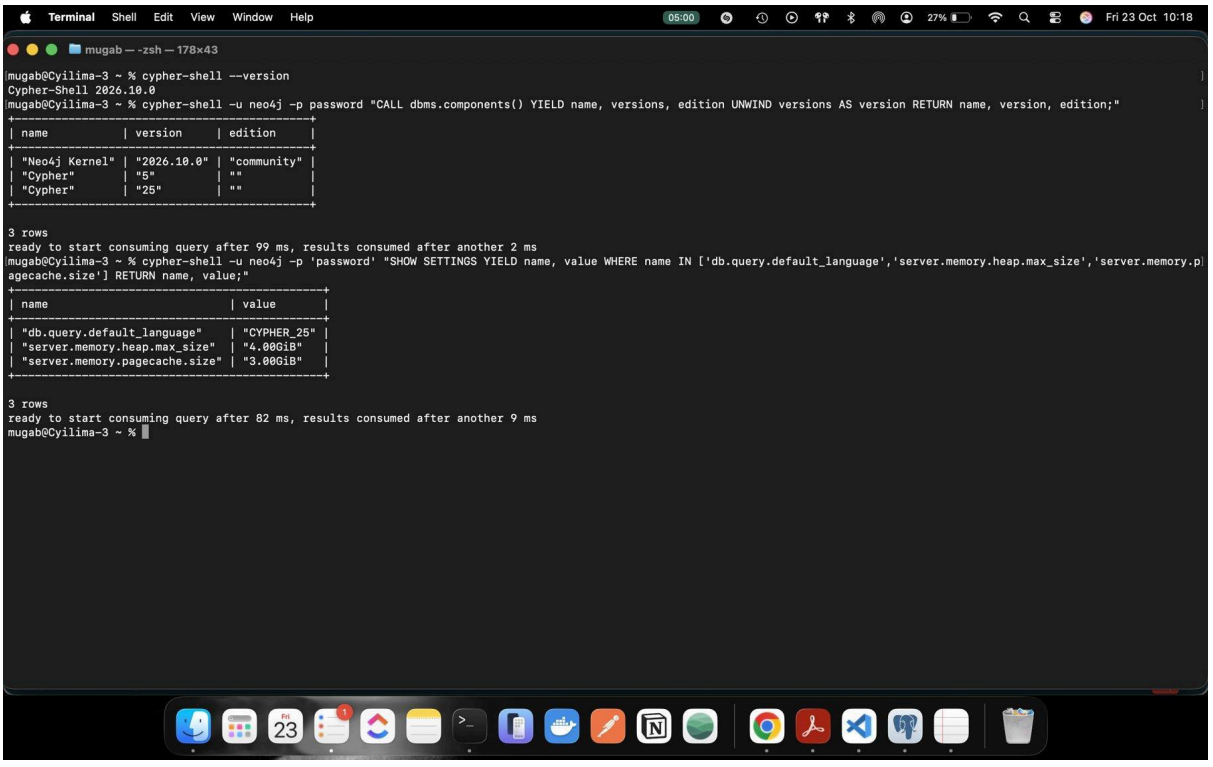


Figure 7: cypher-shell version and available Cypher versions 5 and 25

1.2 Capture the generator streams

capture.py reads each generator stream once, writes it to a CSV in data/, and counts the rows it consumed and wrote against the expected number. For transfers it also keeps an

audit (count, first and last time, total amount) so we can check the databases later against exactly the same numbers.

Table 4: Capture counts (results/capture_summary.json)

Stream	Rows	Seconds	GB	Match
customers	228,000	0.21	0.0036	yes
accounts	248,000	0.53	0.0052	yes
devices	228,600	0.23	0.0036	yes
account_devices	285,600	0.48	0.0063	yes
transfers	20,003,300	46.44	0.7802	yes
Total	20,993,500	47.88	0.7988	yes

Transfer audit: first occurred_at 1788235200 (1 Sep 2026 06:00:00 CAT), last 1790798399 (30 Sep 2026 21:59:59 CAT), total 231,482,327,870 RWF.

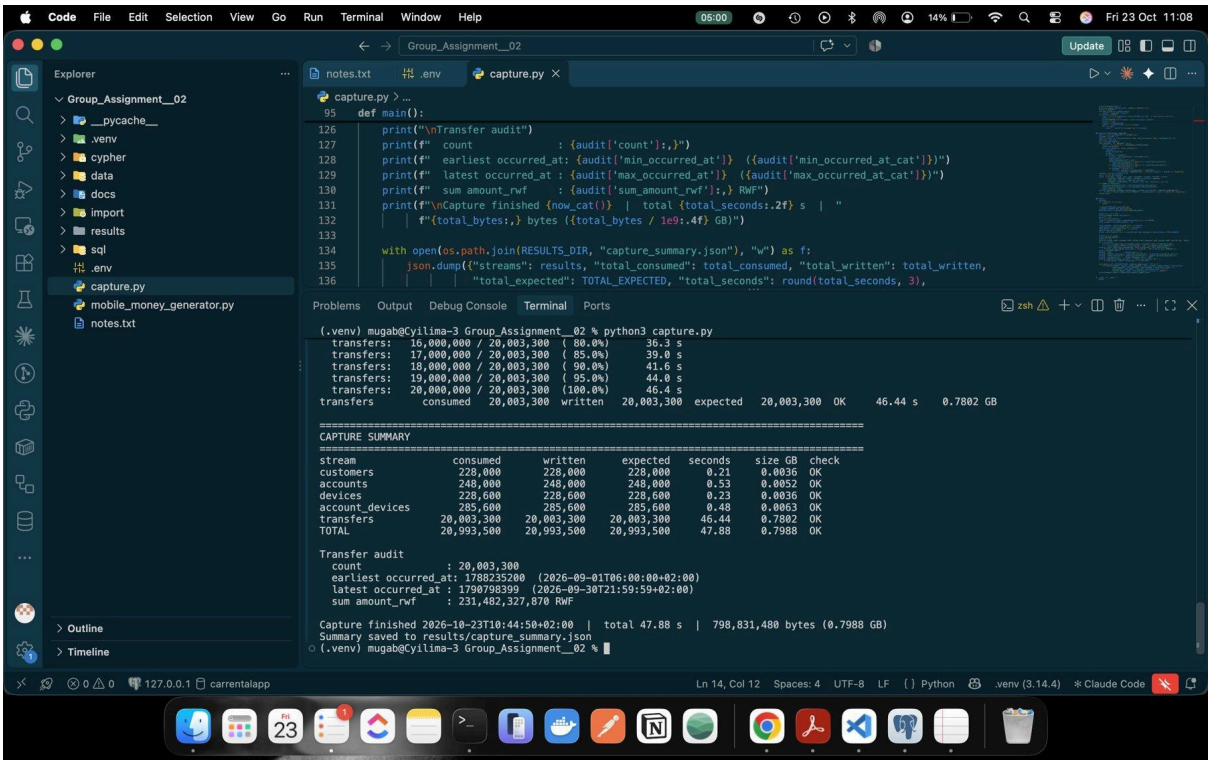


Figure 8: capture.py output: all five streams match and the transfer audit

2 PostgreSQL model

2.1 Schema

We made one table per stream (sql/01_schema.sql). Entity IDs are INTEGER because the largest ID is under 250,000. transfer_id and occurred_at are BIGINT: transfer IDs go above 20 million and Unix seconds are kept as they come from the generator, so no time zone conversion happens at load time. first_day and last_day are SMALLINT, role and access_type are TEXT, and amounts and observations are INTEGER (the largest single transfer fits easily, and we cast to BIGINT when summing).

Row checks stop impossible values from loading: role must be person, merchant or agent; access_type must be one of the device types; observations > 0; 0 ≤ first_day ≤ last_day ≤ 29; and amount_rwf > 0. Primary and foreign keys are added after loading (sql/02_keys.sql) because building them on 20 million rows at once is faster than checking every row during COPY.

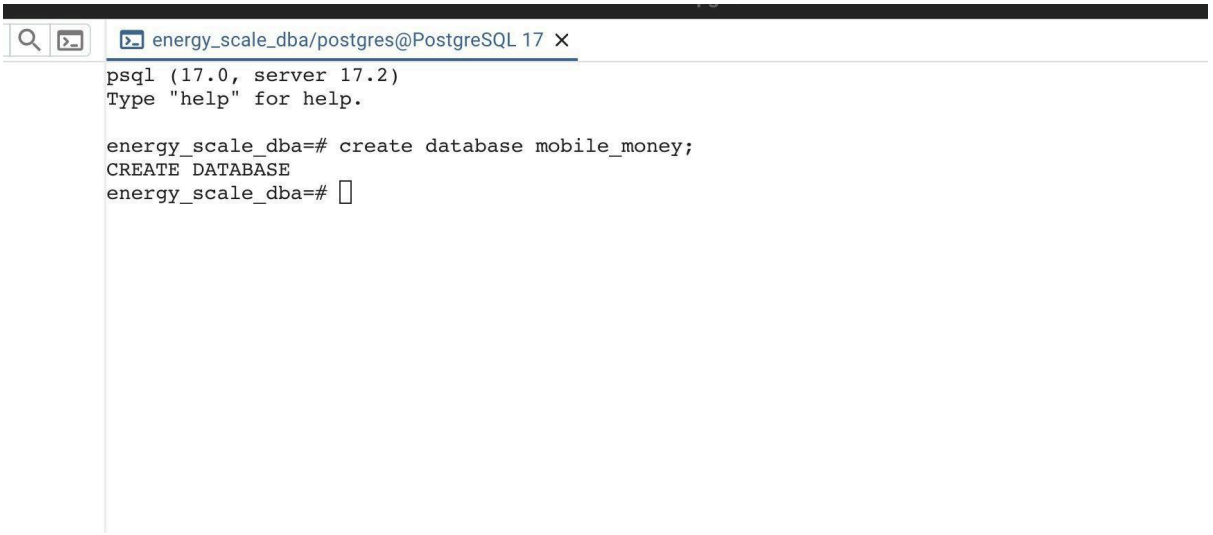


Figure 9: Creating the mobile_money database

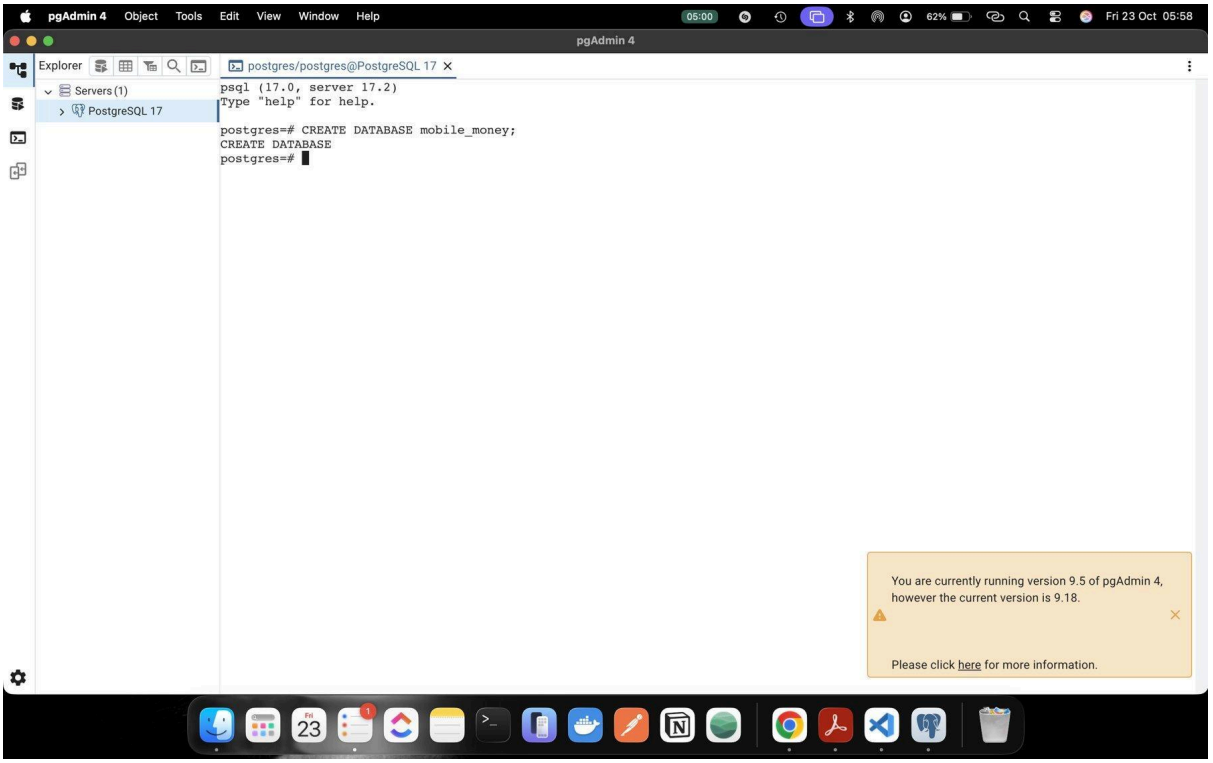


Figure 10: mobile_money database created in pgAdmin's psql tool

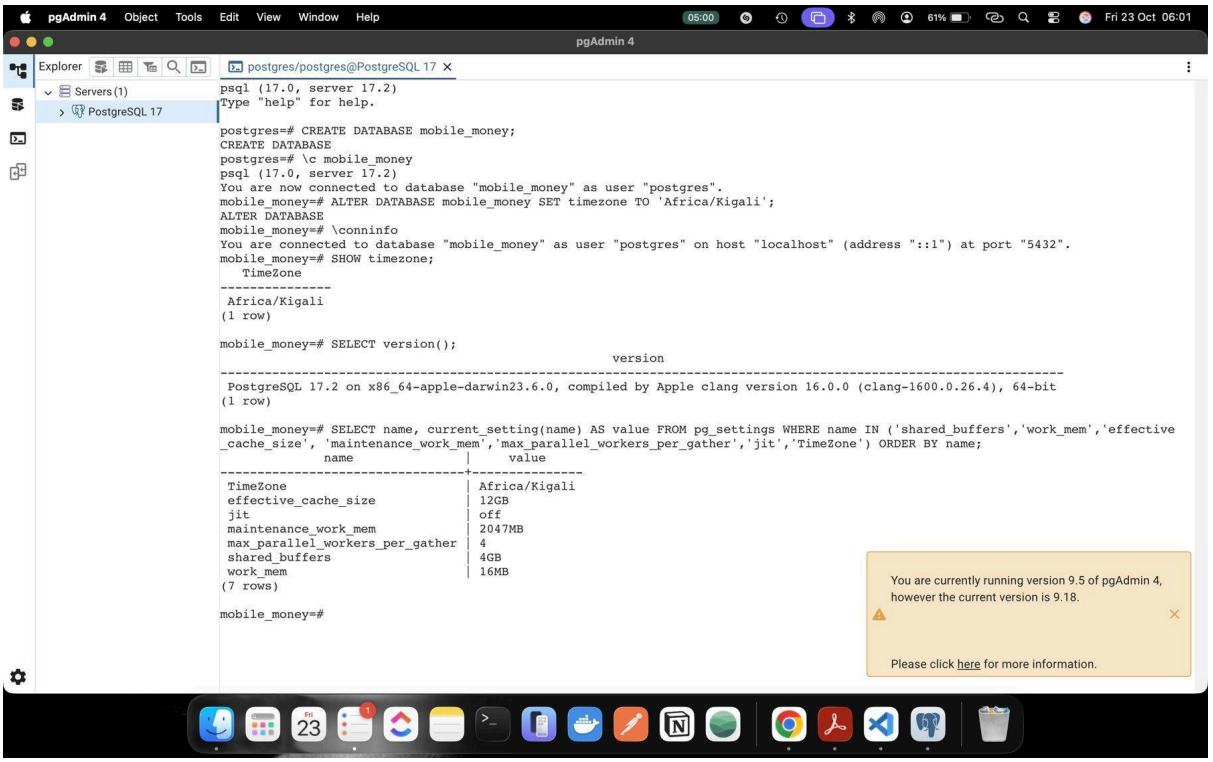


Figure 11: Database time zone set to Africa/Kigali

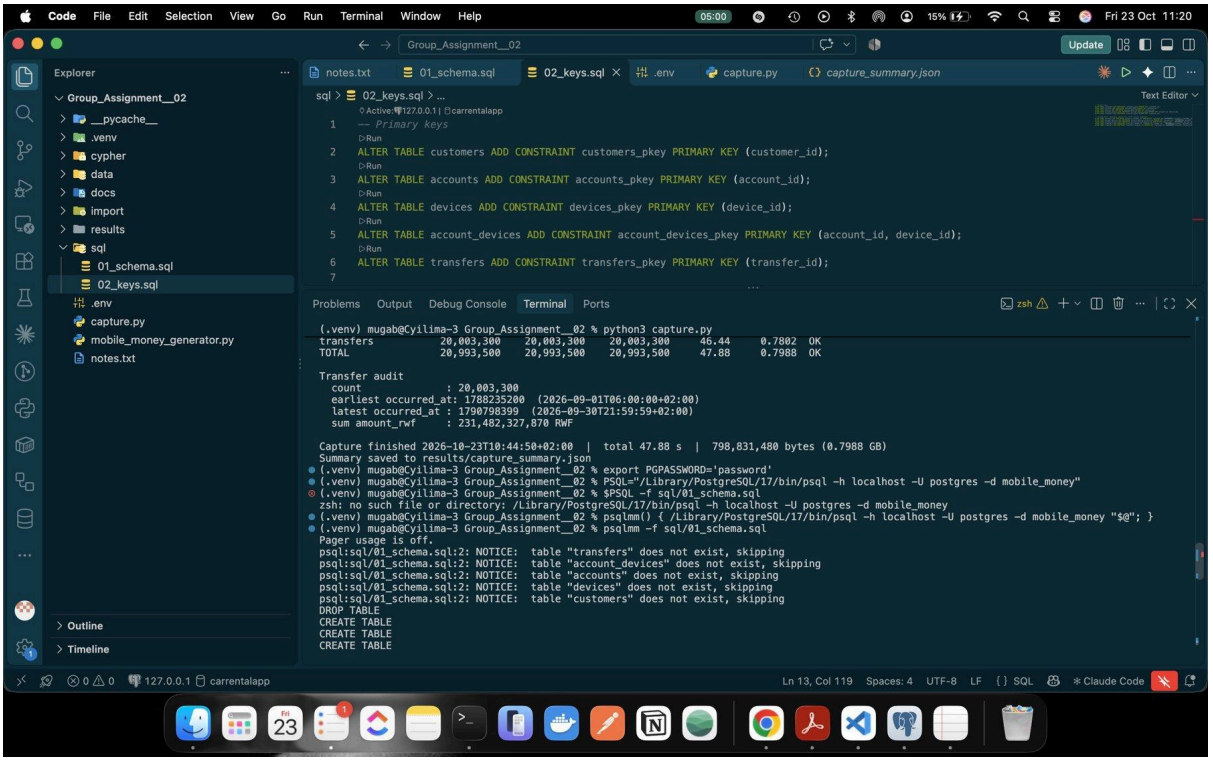


Figure 12: Schema script run: five tables created

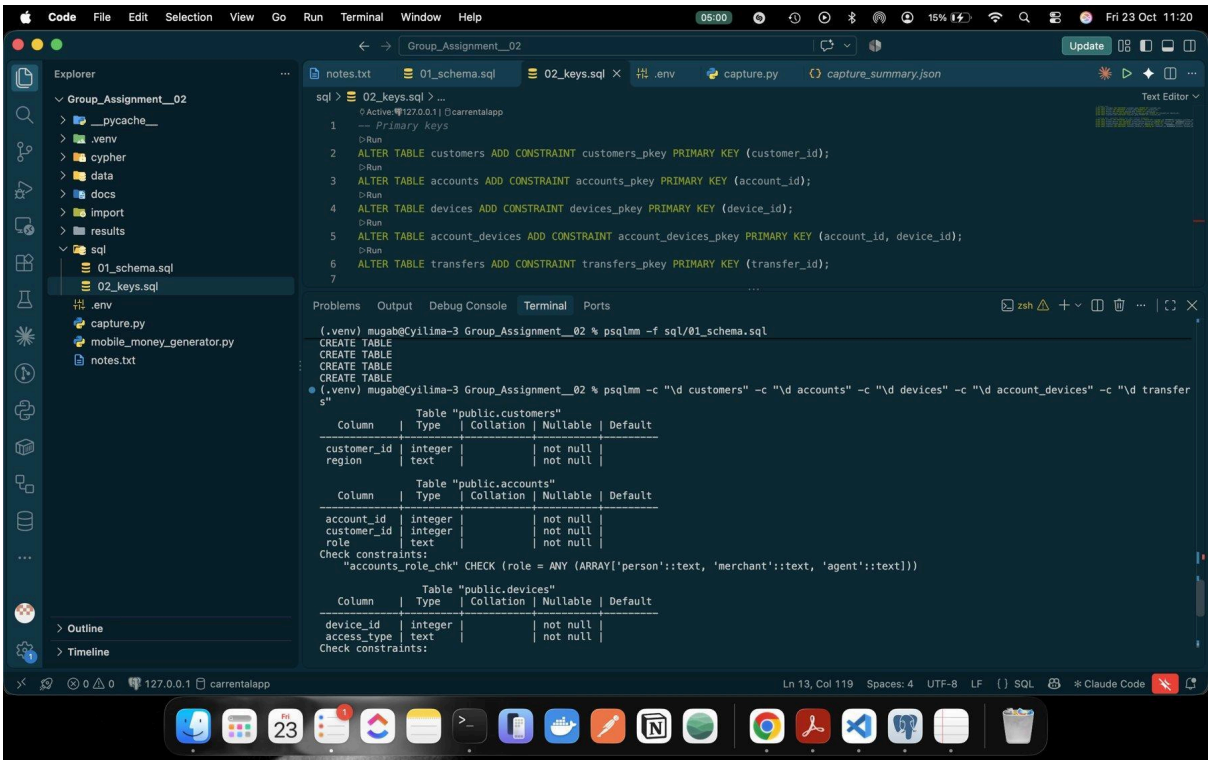


Figure 13: Table definitions with check constraints (accounts, devices)

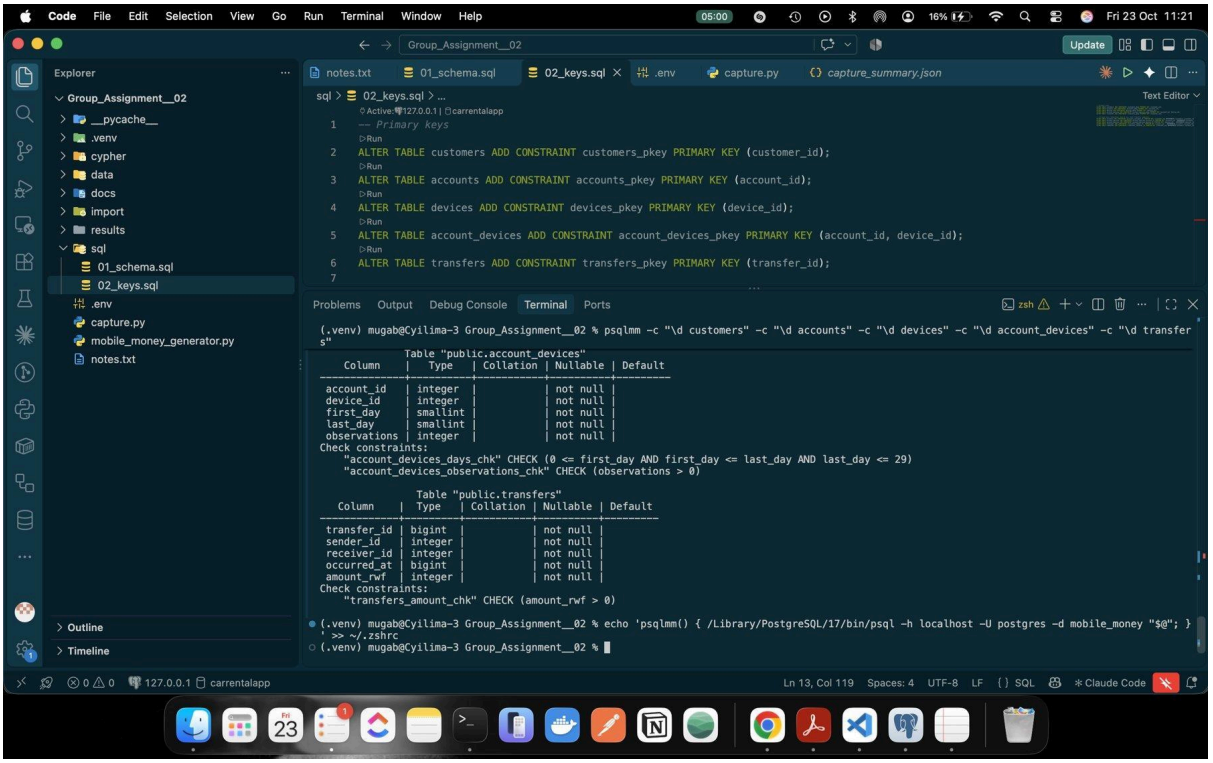


Figure 14: Transfers table: BIGINT transfer_id and occurred_at, amount check

2.2 Load, keys and indexes

We loaded with \copy in dependency order (customers, devices, accounts, account_devices, transfers), then added the keys, five extra indexes and ran ANALYZE. Load took **11.03 s** (transfers alone 9.85 s) and setup took **44.86 s**. The two transfer indexes are covering indexes, (sender_id, occurred_at) and (receiver_id, occurred_at) with the other columns INCLUDEd, so the traversal questions can be answered from the index without visiting the table. A BRIN index on occurred_at helps time-range scans.

Table 5: PostgreSQL storage (GB = 10^9 bytes); whole database 3.7261 GB

Object	Data GB	Index GB	Total GB
transfers	1.2057	2.4335	3.6392
account_devices	0.0129	0.0129	0.0258
accounts	0.0113	0.0112	0.0225
customers	0.0103	0.0051	0.0154
devices	0.0103	0.0052	0.0154
All five tables	1.2504	2.4679	3.7183

All row counts and the transfer audit matched capture.py, roles came out as 240,000 person, 6,000 merchant and 2,000 agent, and all 15 constraints are validated. The indexes are about twice the size of the data, nearly all from the two covering transfer indexes: this is the price we pay for fast lookups.

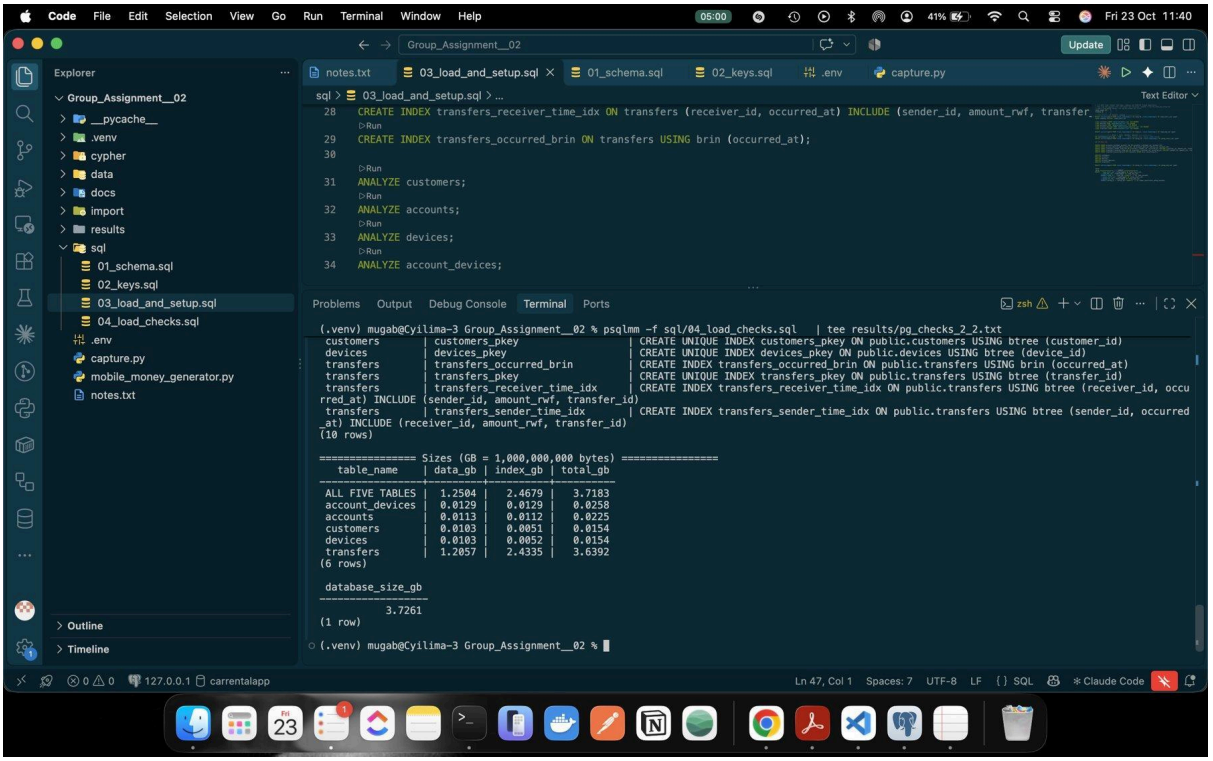


Figure 15: PostgreSQL storage per table and database size

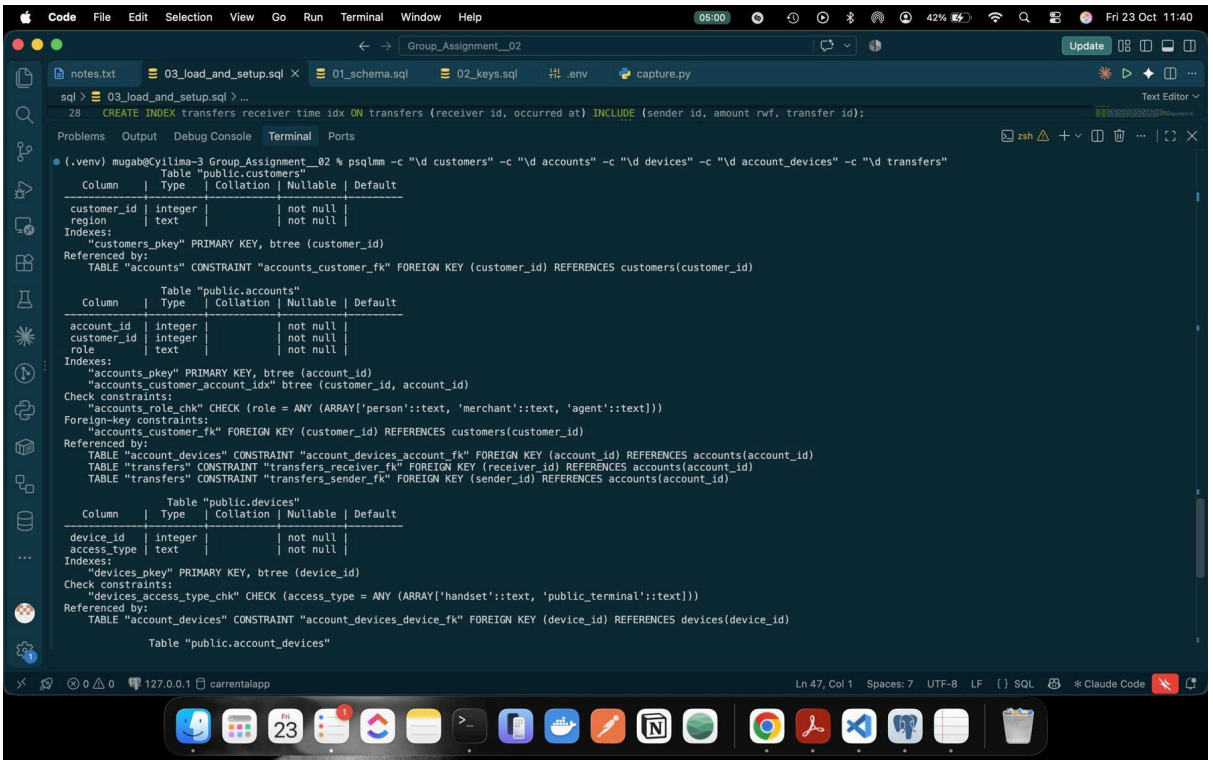


Figure 16: Table definitions after keys and indexes (part 1)

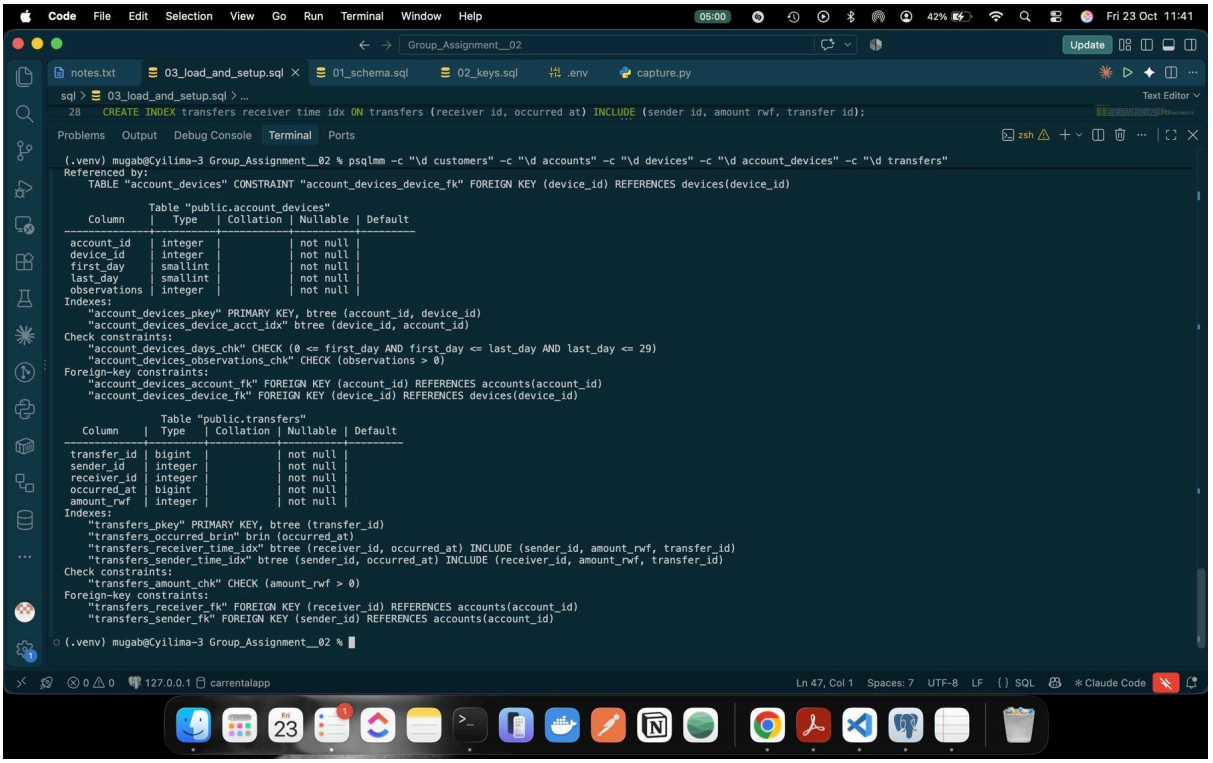


Figure 17: Table definitions after keys and indexes (part 2)

3 Neo4j model

3.1 Graph schema

Each table with its own ID becomes a node label: Customer, Account (with role) and Device (with access_type). The foreign keys become relationships, so the link is stored once instead of being matched by value at query time:

- accounts.customer_id → (Customer)-[:OWNS]->(Account)
- account_devices row → (Account)-[:USED_DEVICE {first_day, last_day, observations}]->(Device)
- transfers.sender_id / receiver_id → (Account)-[:TRANSFER {transfer_id, occurred_at, amount_rwf}]->(Account)

Every transfer is kept as its own TRANSFER relationship. We did not merge repeated payments between the same two accounts into one edge, because Q5 and Q6 need each payment's own time and amount.

3.2 Bulk import

prepare_import.py writes typed-header copies of the CSVs into import/ (the source CSVs stay unchanged). customer_id in accounts.csv is marked :IGNORE and a separate owns.csv (sorted by customer) creates the OWNS relationships. We used neo4j-admin database import full with --id-type=integer.

Table 6: Neo4j import and storage

Measure	Value
Import time	22.723 s (26.1 s wall clock), peak memory 1.014 GiB
Imported	704,600 nodes, 20,536,900 relationships, 62,275,900 properties
Constraints and indexes	17.889 s (3 uniqueness constraints, range index on TRANSFER.occurred_at)
Store size after indexes	2,055,741,440 bytes = 2.06 GB (1.91 GiB)
Transaction log	16 KB: the bulk importer writes the store directly

All import checks passed: node and relationship counts equal the PostgreSQL rows, every account has exactly one owner, roles match and transfer IDs are unique. The import copies were deleted afterwards.

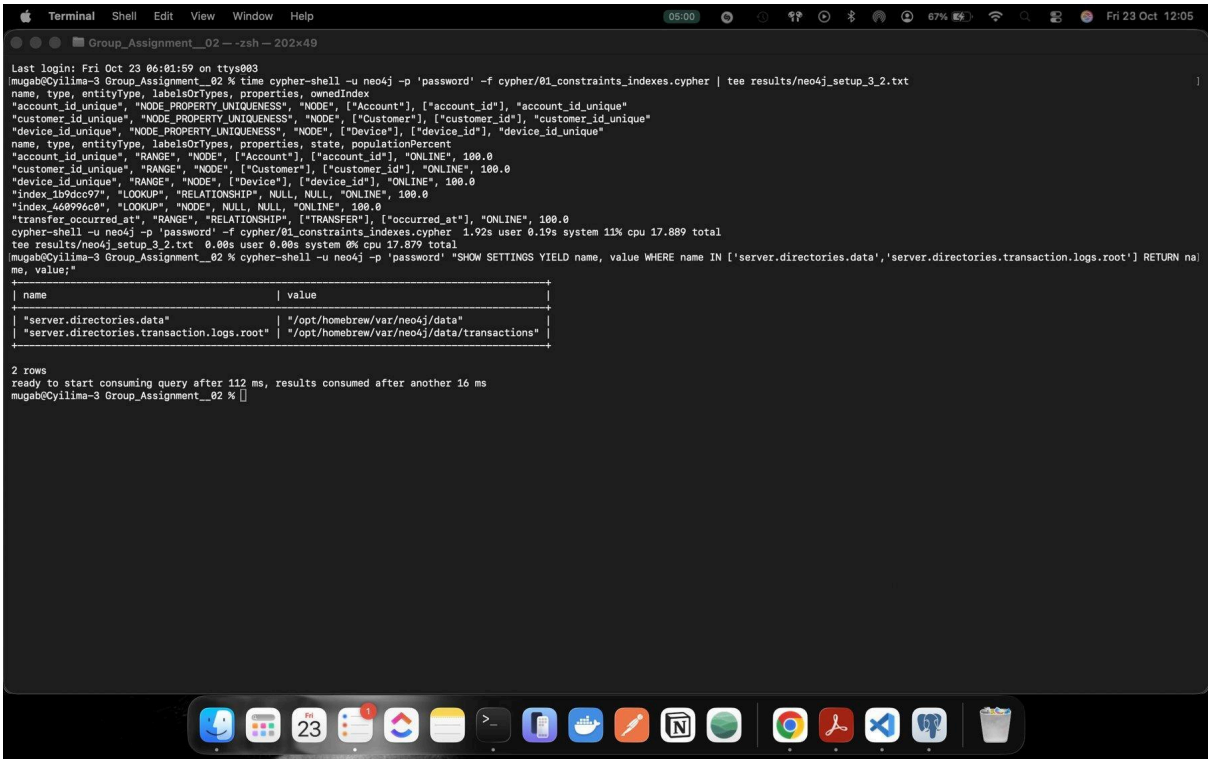


Figure 18: Neo4j constraints and indexes, all ONLINE

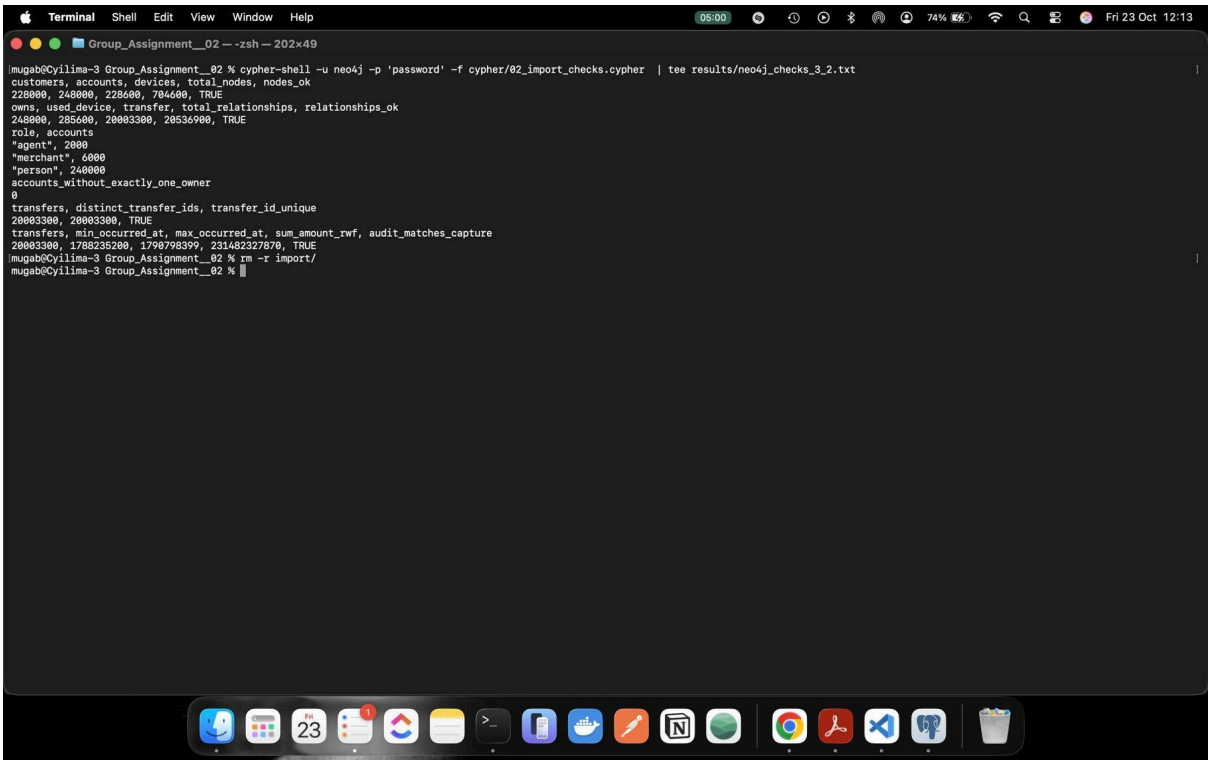


Figure 19: Neo4j import checks: counts, roles, one owner per account

3.3 Trace account 998 in both models

Both models returned the same two rows: account 998 is a person in Kigali owned by customer 998, seen on its own handset 998 (39 observations) and on handset 228001

(16 observations), both over days 0 to 29. Its first outgoing transfer was 113540, 2,100 RWF to merchant 240004 on 1 Sep at 08:43:29 CAT.

In SQL this needs JOINs that match customer_id, account_id and device_id values. In Neo4j the same facts are relationships that the query just follows. Account 998 also paid the same receivers many times, for example 9 payments to account 1111 (117,700 RWF in total). If those were collapsed into one edge with a total, we would lose each payment's ID, time and amount, and with them the order and gaps that Q5 and Q6 depend on.

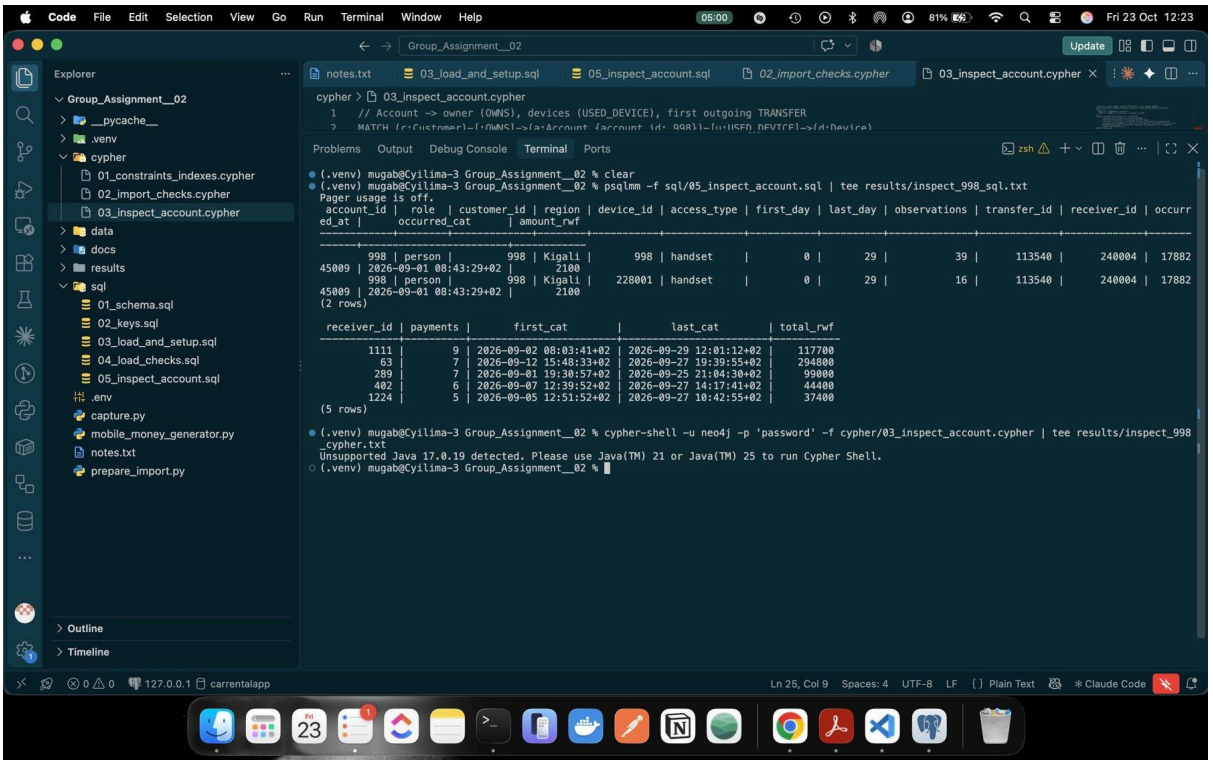


Figure 20: Account 998 traced with SQL

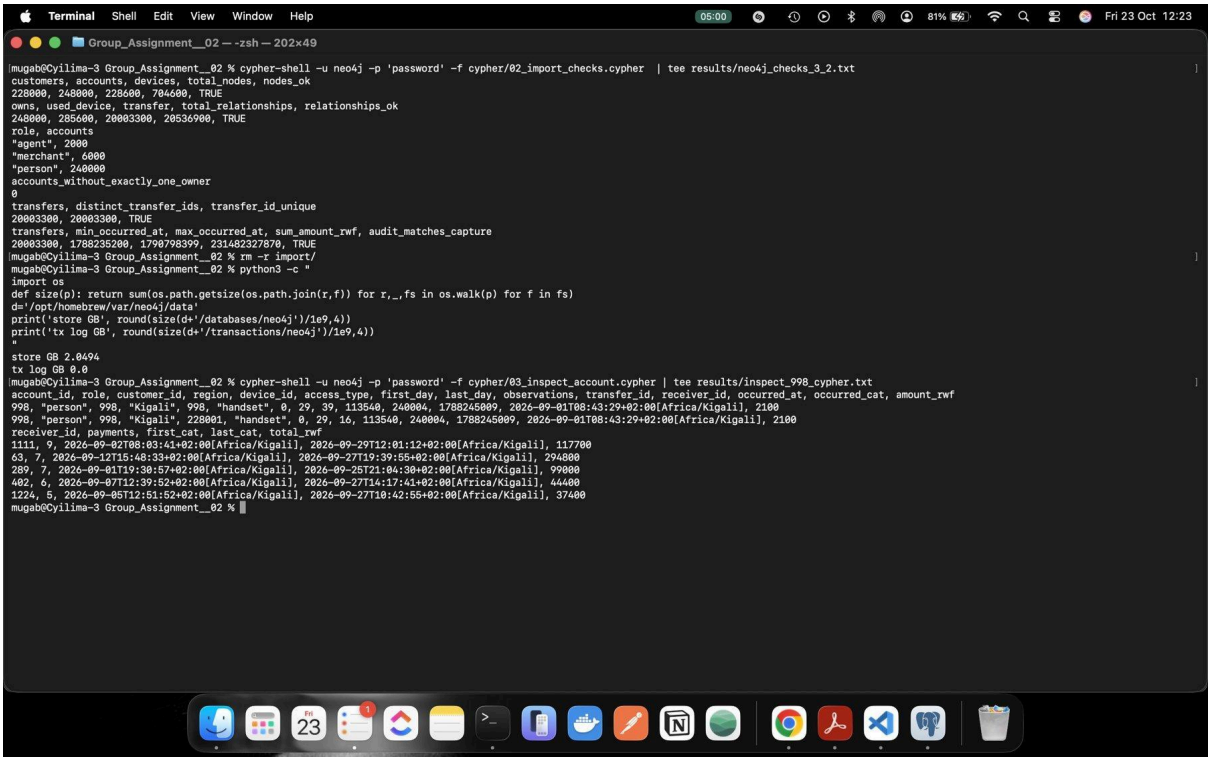


Figure 21: Account 998 traced with Cypher (same rows)

4 Routine business questions

All question code is shared: cases.py holds the six review sets, db.py has run_pg() and run_neo() with persistent connections, and queries.py has every SQL/Cypher pair. CAT hour boundaries are turned into Unix seconds in Python with zone info and passed as parameters.

Table 7: Review sets

Set	Review account	Destination	Date (Sep 2026)
A	998	24755	6
B	147228	186823	13
C	206978	6573	8
D	237998	21755	6
E	144228	183823	13
F	203978	3573	8

4.1 Q1 Account activity

Q1 counts each review account's outgoing transfers and their value from 09:45:00 (included) to 10:45:00 (excluded) CAT, for all receiver roles. Both queries return a row of zeros when nothing matches.

Table 8: Q1 results

Set	Account	Window start / end (epoch)	Transfers	Total RWF	SQL = Cypher
A	998	1788680700 / 1788684300	1	200,000	yes
B	147228	1789285500 / 1789289100	1	1,050,000	yes
C	206978	1788853500 / 1788857100	1	300,000	yes
D	237998	1788680700 / 1788684300	1	45,000	yes
E	144228	1789285500 / 1789289100	1	40,000	yes
F	203978	1788853500 / 1788857100	1	30,000	yes

Each account sent one transfer in its hour, but A, B and C sent 200,000 to 1,050,000 RWF while D, E and F sent 30,000 to 45,000 RWF on the same dates. **Outgoing value is not a balance**: a balance would also need the opening balance, incoming transfers, cash-in and cash-out at agents, fees and reversals, and these records are not a full ledger. B sent 1,050,000 RWF but could have received the same amount a minute earlier.

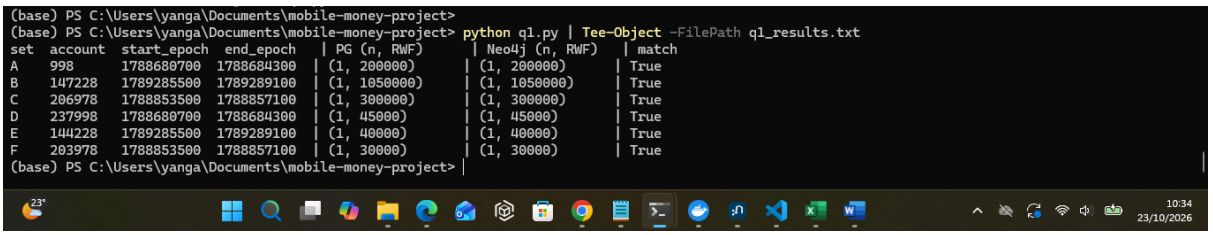


Figure 22: Q1.py results

4.2 Q2 Daily activity

Q2 groups all transfers by CAT calendar day: in SQL with (to_timestamp(occurred_at) AT TIME ZONE 'Africa/Kigali')::date and in Cypher with date(datetime({epochSeconds: …, timezone: 'Africa/Kigali'})). Both returned the same 30 rows; the daily counts add up to 20,003,300 and the daily totals to 231,482,327,870 RWF, exactly the capture numbers.

The count is almost flat at about 666,667 per day because the background transfers are spread evenly. From 6 to 15 September there are 150 to 330 extra transfers per day and clearly higher value (about 7.74 to 7.86 billion RWF, highest 7,859,428,355 RWF on 15 Sep, against 7.66 to 7.70 billion on most days). These are the large, quick payments like those in Q1. **Value is not revenue**: the money belongs to customers; the provider earns fees and commissions, which are not in the data.

```
(.venv) mugab@Cyilima-3 Group_Assignment__02 % python3 run_q2.py | tee results/q2_results.txt
Q2: transfers and total RWF per CAT day (September 2026)

day           SQL count    SQL total RWF  Cypher count  Cypher total RWF  match
------------------------------------------------------------------------------
2026-09-01      666,667    7,683,326,000       666,667     7,683,326,000  YES
2026-09-02      666,667    7,678,846,900       666,667     7,678,846,900  YES
2026-09-03      666,666    7,665,877,900       666,666     7,665,877,900  YES
2026-09-04      666,667    7,665,199,000       666,667     7,665,199,000  YES
2026-09-05      666,667    7,679,091,900       666,667     7,679,091,900  YES
2026-09-06      666,816    7,741,994,180       666,816     7,741,994,180  YES
2026-09-07      666,817    7,726,576,180       666,817     7,726,576,180  YES
2026-09-08      666,907    7,777,047,990       666,907     7,777,047,990  YES
2026-09-09      666,906    7,783,400,210       666,906     7,783,400,210  YES
2026-09-10      666,997    7,807,170,170       666,997     7,807,170,170  YES
2026-09-11      666,847    7,746,125,100       666,847     7,746,125,100  YES
2026-09-12      666,936    7,783,593,400       666,936     7,783,593,400  YES
2026-09-13      666,937    7,800,995,510       666,937     7,800,995,510  YES
2026-09-14      666,997    7,856,644,775       666,997     7,856,644,775  YES
2026-09-15      666,996    7,859,428,355       666,996     7,859,428,355  YES
2026-09-16      666,787    7,683,087,200       666,787     7,683,087,200  YES
2026-09-17      666,757    7,699,710,100       666,757     7,699,710,100  YES
2026-09-18      666,786    7,677,843,700       666,786     7,677,843,700  YES
2026-09-19      666,757    7,665,841,400       666,757     7,665,841,400  YES
2026-09-20      666,757    7,663,404,900       666,757     7,663,404,900  YES
2026-09-21      666,726    7,696,315,800       666,726     7,696,315,800  YES
2026-09-22      666,727    7,703,220,100       666,727     7,703,220,100  YES
2026-09-23      666,697    7,697,325,900       666,697     7,697,325,900  YES
2026-09-24      666,726    7,668,636,000       666,726     7,668,636,000  YES
2026-09-25      666,697    7,677,231,000       666,697     7,677,231,000  YES
2026-09-26      666,697    7,671,715,700       666,697     7,671,715,700  YES
2026-09-27      666,696    7,692,655,900       666,696     7,692,655,900  YES
2026-09-28      666,667    7,676,184,700       666,667     7,676,184,700  YES
2026-09-29      666,667    7,668,892,500       666,667     7,668,892,500  YES
2026-09-30      666,666    7,684,945,400       666,666     7,684,945,400  YES

Checks
  days returned        : SQL 30, Cypher 30 (expected 30)
  all rows identical   : True
  sum of daily counts  : 20,003,300  (capture 20,003,300)  -> True
  sum of daily totals  : 231,482,327,870 RWF  (capture 231,482,327,870)  -> True
  query time (single run, not the 6.2 benchmark): SQL 1.81 s, Cypher 45.19 s
```

Figure 22: Q2 output: 30 identical days, totals reconcile to the capture

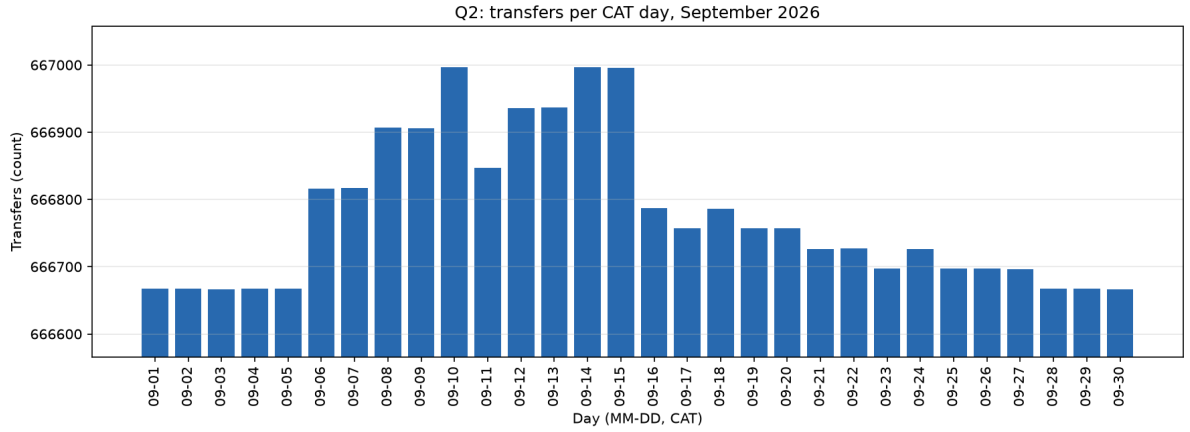


Figure 23: Q2 transfers per CAT day (y-axis starts above zero)

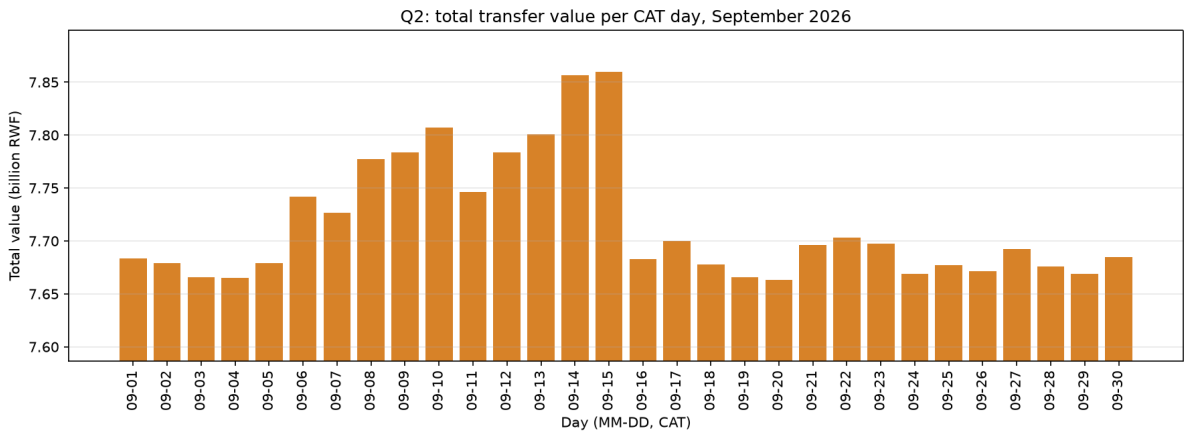


Figure 24: Q2 total transfer value per CAT day (y-axis starts above zero)

4.3 Q3 Shared devices

Q3 self-joins account_devices on device_id (SQL) or matches Account → Device ← Account (Cypher), keeping other accounts whose observed interval overlaps: max(first days) ≤ min(last days). Cypher has no GREATEST/LEAST, so CASE gives the same values. Both engines returned identical rows: 4, 9, 9, 4, 9 and 9 (44 in total).

Table 9: Q3 shared devices (bold = the set's Q7 destination)

Set	Rows	Shared devices
A	4	998 → 220998 (second wallet of customer 998); 228001 → 8917, 16836, 24755
B	9	147226 → 147226, 147227, 147229, 147230; 228018 → 155147, 163066, 170985, 178904, 186823
C	9	206976 → 206976 (from day 11), 206977, 206979, 206980; 228043 → 6573, 214897, 222816, 230735, 238654
D	4	17998 → 17998; 228301 → 5917, 13836, 21755
E	9	144226 → 144226, 144227, 144229, 144230; 228318 → 152147, 160066, 167985, 175904, 183823
F	9	203976 → 203976 (from day 11), 203977, 203979, 203980; 228343 → 3573, 211897, 219816, 227735, 235654

Every account has a household-style phone and a second handset shared with 3 to 5 accounts, and each set's destination is on that second handset. A legitimate reason: in many Rwandan homes one smartphone serves several family members with their own wallets, and a person can keep a personal and a business wallet on one phone (998 and 220998). **Overlap is not simultaneous use**: the interval is only the first and last day the account was seen on the device, there are no usage times, and the data does not show which device made a payment.

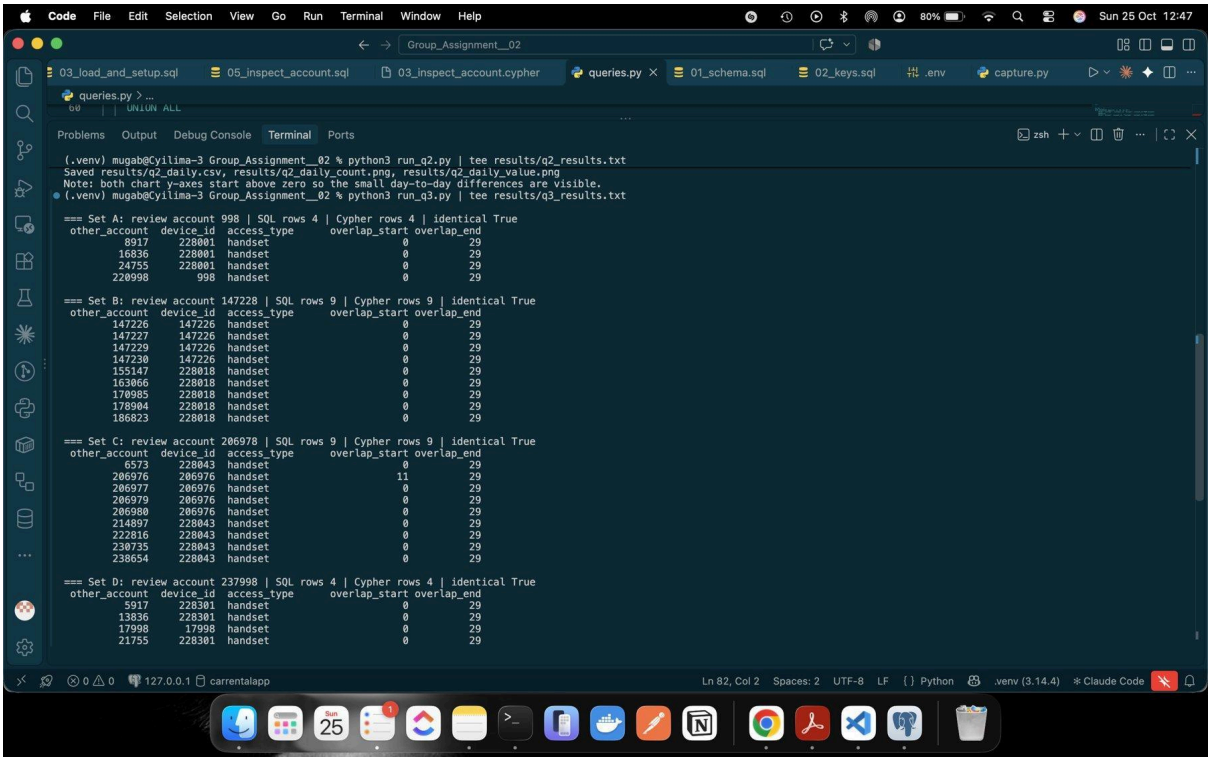


Figure 25: Q3 output (sets A to C)

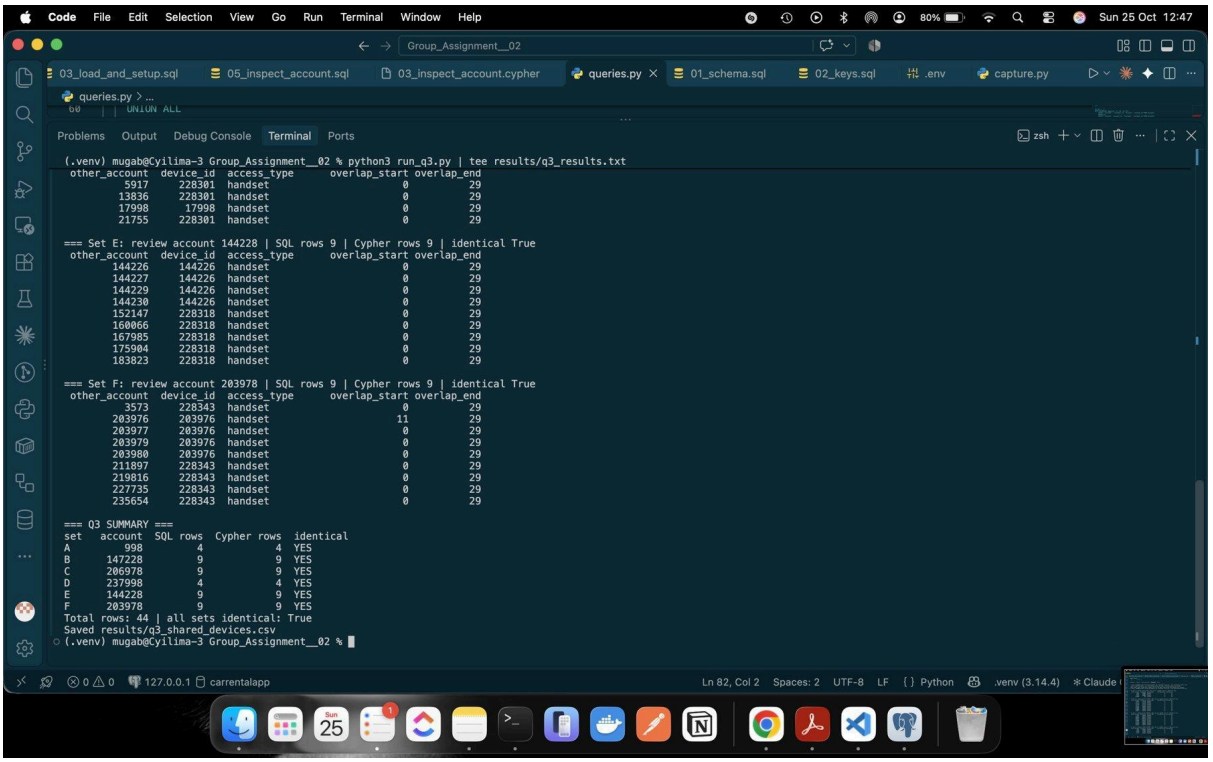


Figure 26: Q3 output (sets D to F and summary)

5 Network and investigation questions

5.1 Q4 Ownership and device network

Q4 follows ownership and device links in both directions up to 4, 6, 8 and 10 links and counts every reached account once by its owner's region. The SQL needs an edges CTE that carries the node kind with each ID, because Customer 7, Account 7 and Device 7 are different things; a recursive walk then expands up to the depth limit. Cypher uses an undirected [:OWNS|USED_DEVICE*1..d] pattern and WITH DISTINCT b before grouping. All 24 comparisons were identical.

Table 10: Distinct accounts reached (per-region counts in results/q4_all_depths.txt)

Set	Depth 4	Depth 6	Depth 8	Depth 10
A	10	43	175	187
B	46	178	188	228
C	146	483	832	1,433
D	10	44	180	192
E	47	183	193	230
F	147	488	837	1,426

Prediction and result. We did not write the prediction down before running depths 8 and 10, so we only give our reasoning here: each account's close network is a household phone, a shared handset and the other wallets of those owners, so where that neighbourhood is closed the count should rise and then level off, and where it reaches a widely shared device it should keep growing. That is what happened: A, B, D and E flatten (D 180 → 192), while C and F keep growing (483 → 832 → 1,433) because their walks reach devices used by many accounts.

Supporting path (4 links): Account 998 –USED_DEVICE– Device 228001 –USED_DEVICE– Account 8917 –OWNS– Customer 8917 –OWNS– Account 228917. Account 228917 is reached only because its owner shares a phone with 998. At 10 links, 998 is "linked" to 187 accounts in all five regions; most of these links say more about the shape of the network than about any real relationship, so close links matter much more than distant ones.

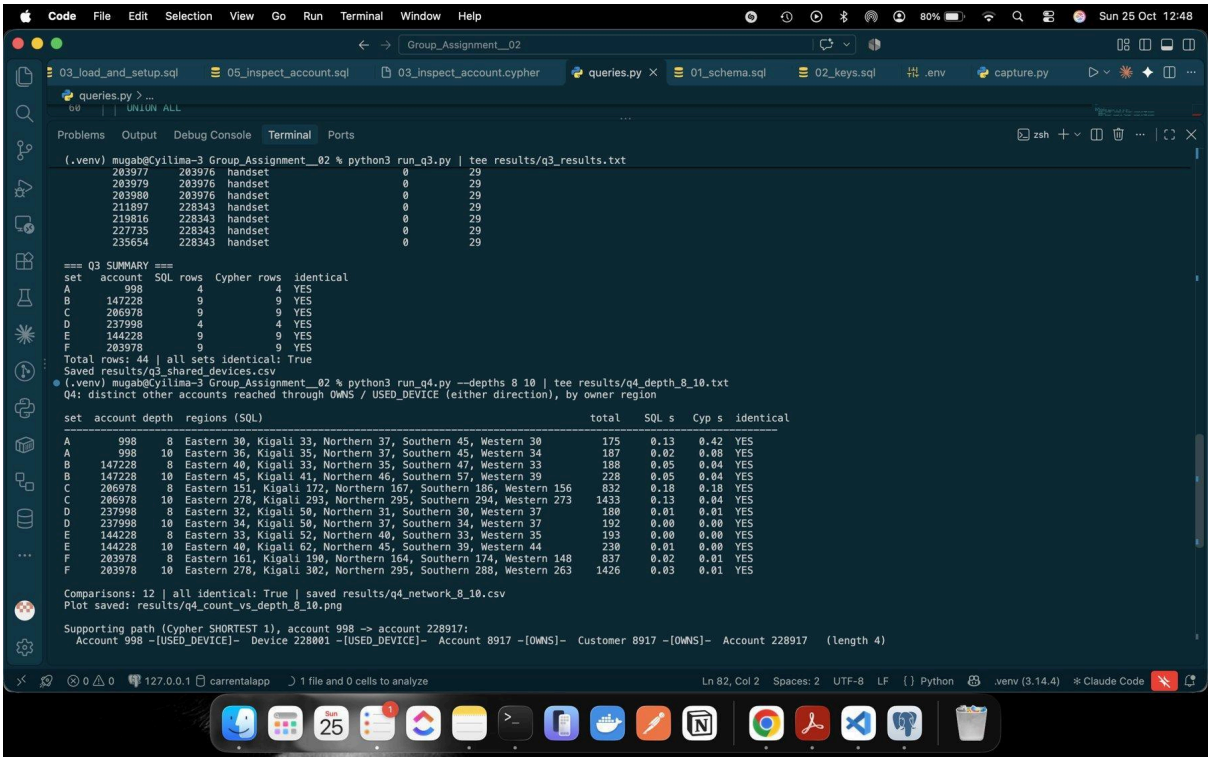


Figure 27: Q4 depths 8 and 10 with the supporting path

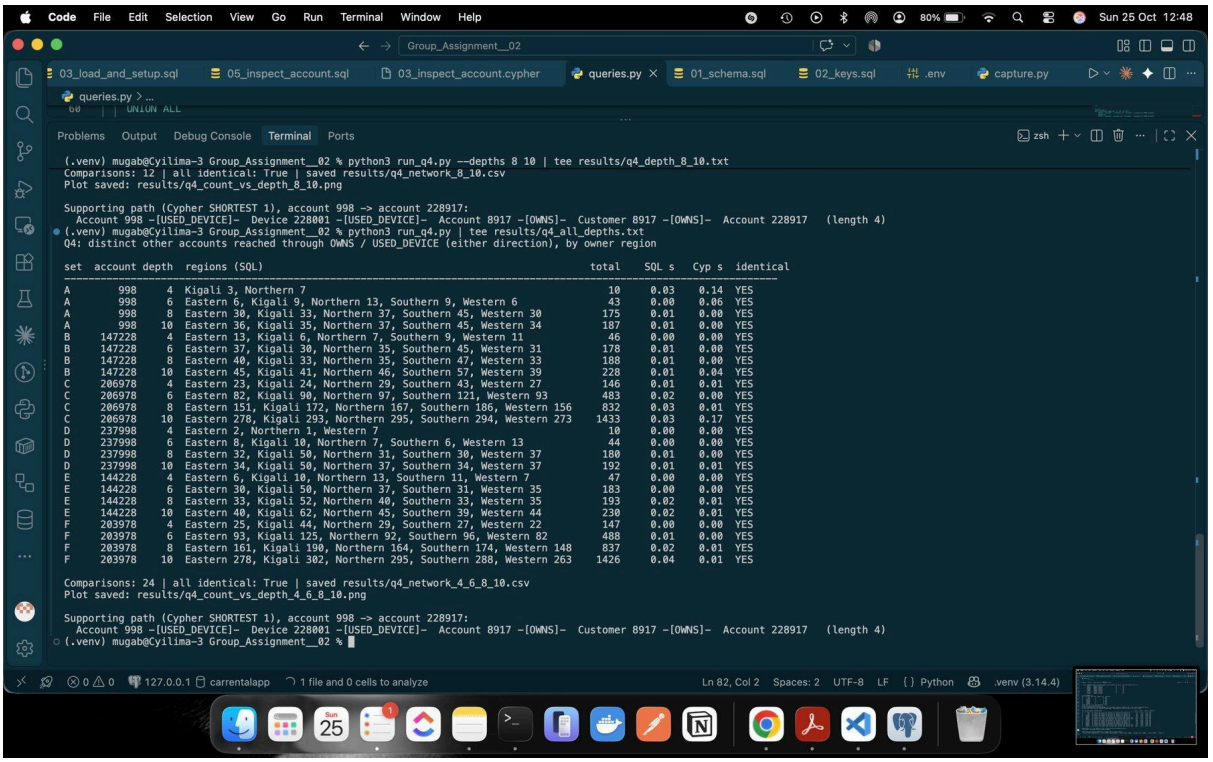


Figure 28: Q4 all depths: 24 comparisons identical

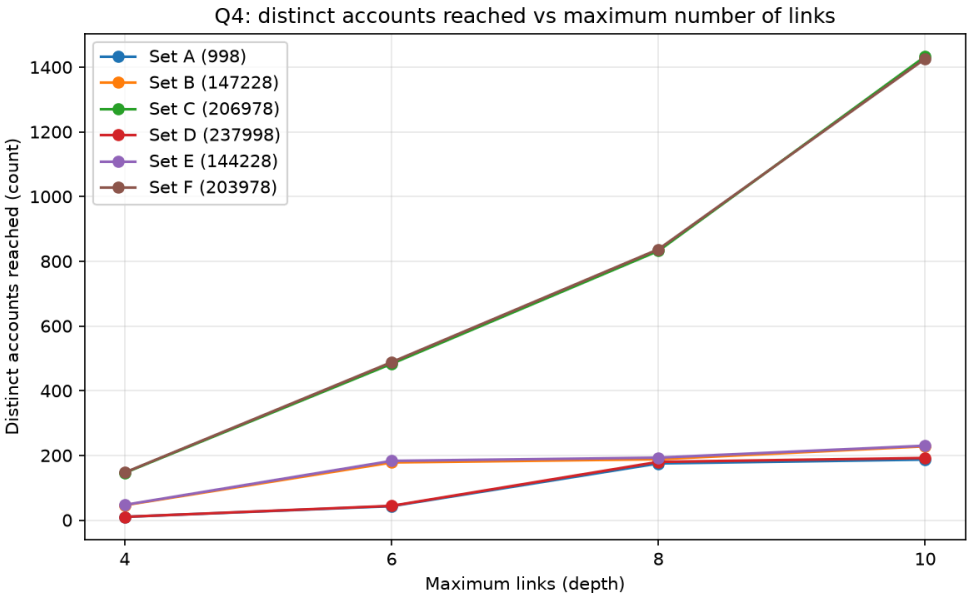


Figure 29: Q4 distinct accounts reached against depth

5.2 Q5 Rapid onward payments

Q5 finds chains of exactly three transfers through four different accounts. The first transfer is in the review hour, each next one is 1 to 600 s after the previous, and each onward amount satisfies 90 × previous ≤ 100 × next ≤ 105 × previous (integers, no rounding).

Table 11: Q5 rapid chains (identical in both engines)

Set	Chains	Evidence
A	1	20000001-3: 998 → 8917 → 16836 → 24755, 6 Sep 10:00:00 / 10:00:40 / 10:01:20; 200,000 → 196,000 → 192,080 RWF
B	1	20000092-4: 147228 → 155147 → 163066 → 170985, 13 Sep from 10:00:17, +40 s each; 1,050,000 → 1,029,000 → 1,008,420 RWF
C	1	20000229-31: 206978 → 214897 → 222816 → 230735, 8 Sep from 10:00:42, +40 s each; 300,000 → 294,000 → 288,120 RWF
D, E, F	0	none

Each step passes on 98% of what came in, 40 seconds later, which fits money being moved quickly with about 2% kept at each hop. It does **not** prove that the same money moved (wallets mix funds), that one person controls the accounts, or any wrongdoing; a family or business group settling a shared payment could look the same.

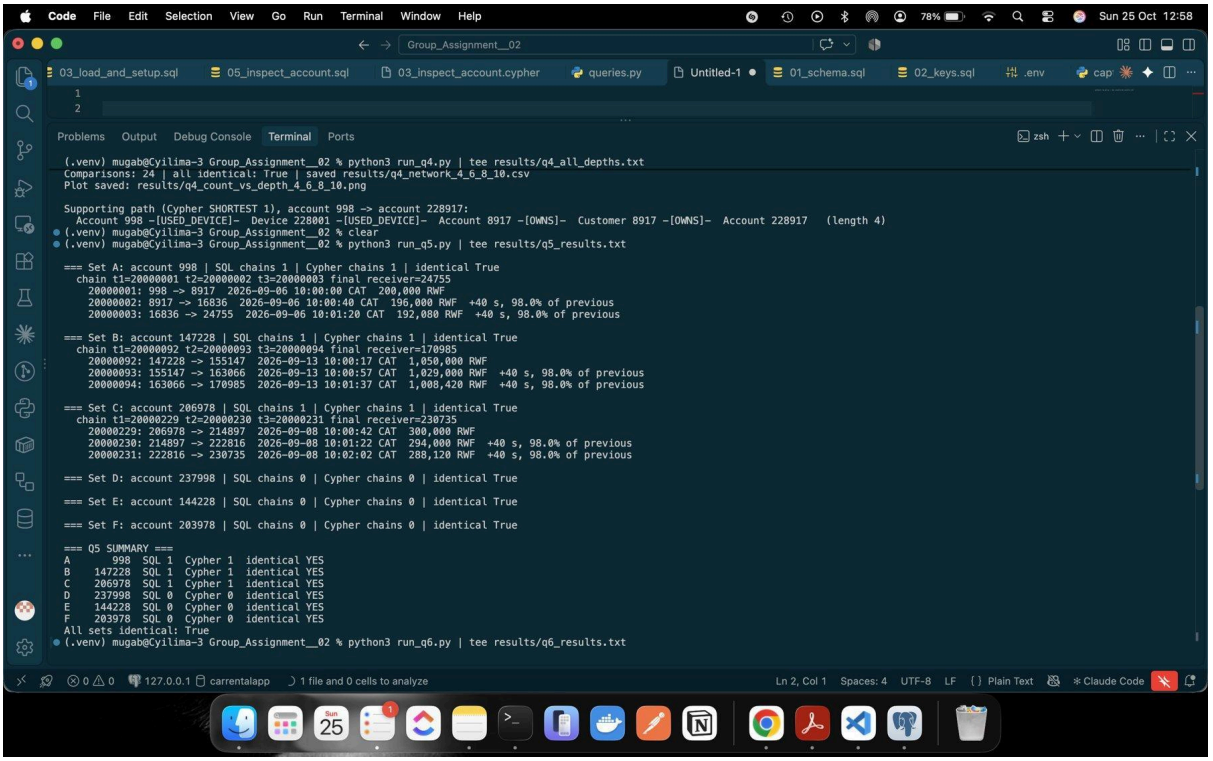


Figure 30: Q5 output with transfer details

5.3 Q6 Circular payments

The recursive SQL carries the path of transfer IDs, current account, first and last time and last amount, extends only eligible transfers and stops at the first return. Cypher uses a bounded quantified path restricted to the hour, then checks time order, the 1,800 s span and the amount rule over consecutive relationships.

Table 12: Q6 cycles (identical in both engines)

Set	Cycles	Evidence
A	1	4 transfers: 998 → 8917 → 16836 → 24755 → 998, 10:00:00 to 10:02:00 (span 120 s); 200,000 → 188,238 RWF
C	1	6 transfers: 206978 → 214897 → 222816 → 230735 → 238654 → 6573 → 206978, 10:00:42 to 10:04:02 (span 200 s); 300,000 → 271,176 RWF
B, D, E, F	0	B's chain moves on but does not come back within the hour

Further evidence we would ask for: KYC records of every account in the loop, the device and location used to start each transfer, balances before and after, the payment descriptions, and whether the same loop happens on other days.

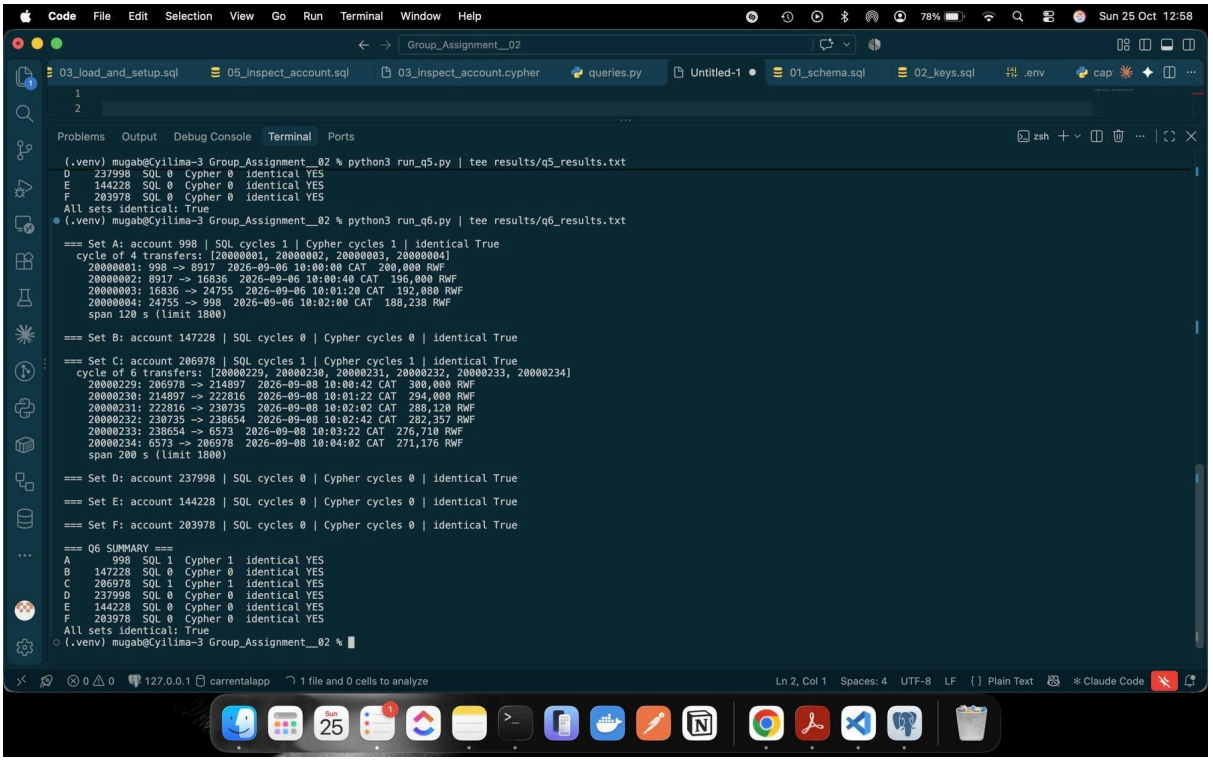


Figure 31: Q6 output with each transfer of the two cycles

5.4 Q7 Shortest person-only path

Q7 looks for the shortest directed route from the review account to its destination over the whole month, through person accounts only, with bounds 4 and 6. The person rule is applied during the search (a role join in each recursive step, or a role predicate inside the Cypher quantified pattern). Each path was checked against stored transfers and roles.

Table 13: Q7 lengths (same in both engines; B paths differ but both are valid)

Set	Route	Bound 4	Bound 6	Path (bound 6)
A	998 → 24755	3	3	998 → 8917 → 16836 → 24755
B	147228 → 186823	none	5	SQL: 147228 → 147793 → 147197 → 148101 → 178904 → 186823; Cypher: 147228 → 148132 → 147536 → 148101 → 178904 → 186823
C	206978 → 6573	none	5	206978 → 214897 → 222816 → 230735 → 238654 → 6573
D	237998 → 21755	3	3	237998 → 5917 → 13836 → 21755
E	144228 → 183823	none	5	144228 → 152147 → 160066 → 167985 → 175904 → 183823
F	203978 → 3573	none	5	203978 → 211897 → 219816 → 227735 → 235654 → 3573

No route within 4 does not mean not connected: B, C, E and F connect in 5. **A route is not a flow of funds**: Q7 ignores time and amount. D reaches its destination in 3

transfers just like A, but D's transfers were 45,000 RWF on 6, 8 and 10 September, so nothing shows money moving along the path.

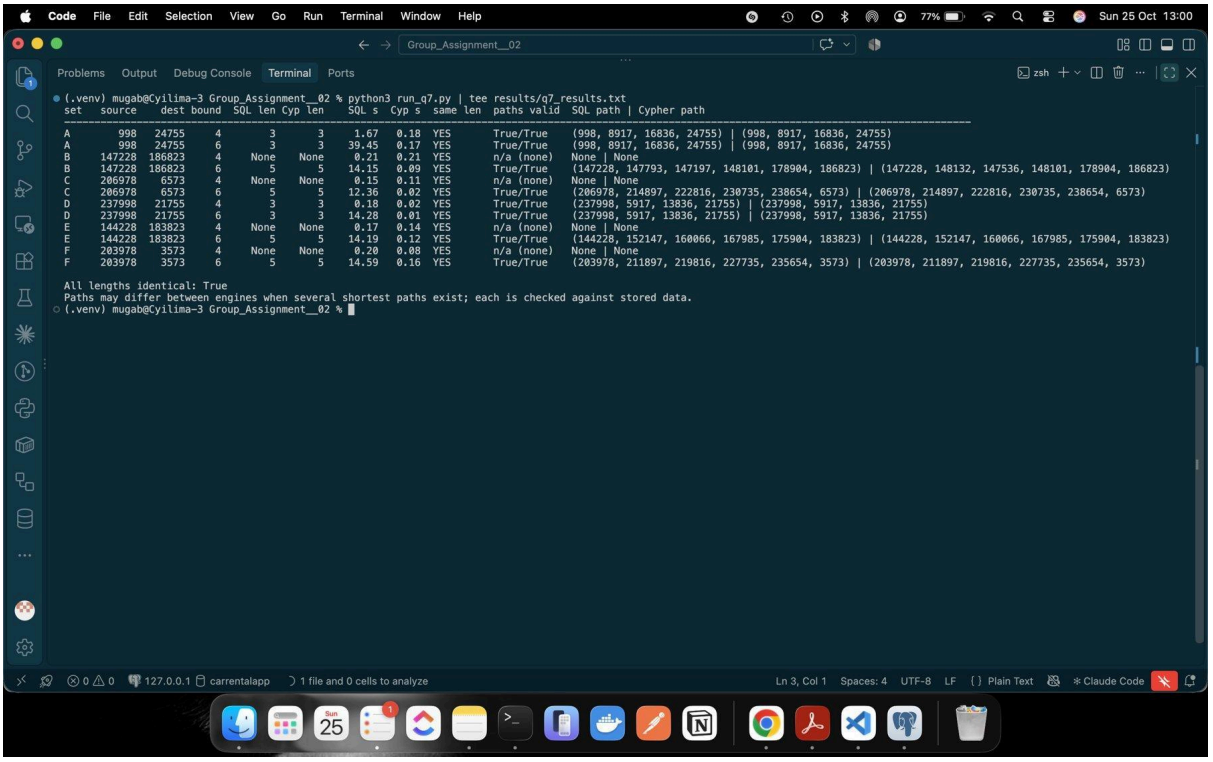


Figure 32: Q7 output: lengths, paths and validity for bounds 4 and 6

6 Correctness and timing

6.1 One comparison program

compare.py runs all 61 cases (Q1 6, Q2 1, Q3 6, Q4 24, Q5 6, Q6 6, Q7 12) through run_pg() and run_neo(). Values are normalised (integers, ISO dates, lists as tuples) and rows are sorted before an exact comparison. Q7 lengths are compared on their own, then each engine's path is checked for endpoints, a stored transfer per step in the right direction, person roles and correct length. **Result: 61/61 match and every Q7 path is valid.**

Two independent checks read the CSVs with Python only. Device overlap: 998 and 24755 were both on handset 228001 over days [0, 29], giving (24755, 228001, handset, 0, 29), the same as the Q3 row. Cycle: transfers 20000001 to 20000004 link correctly, start and end at 998, increase in time inside the hour, span 120 s and each amount is 98% of the previous; all rules confirmed.

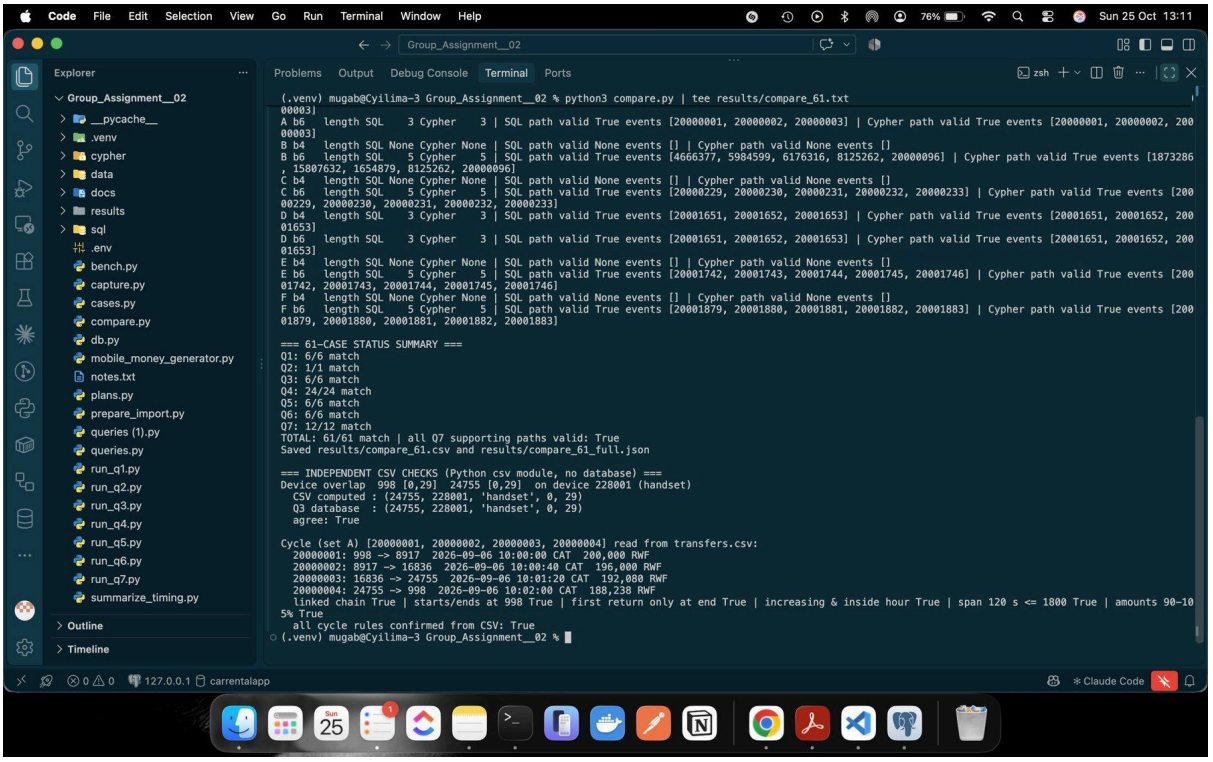


Figure 33: Comparison program: Q7 length and path checks

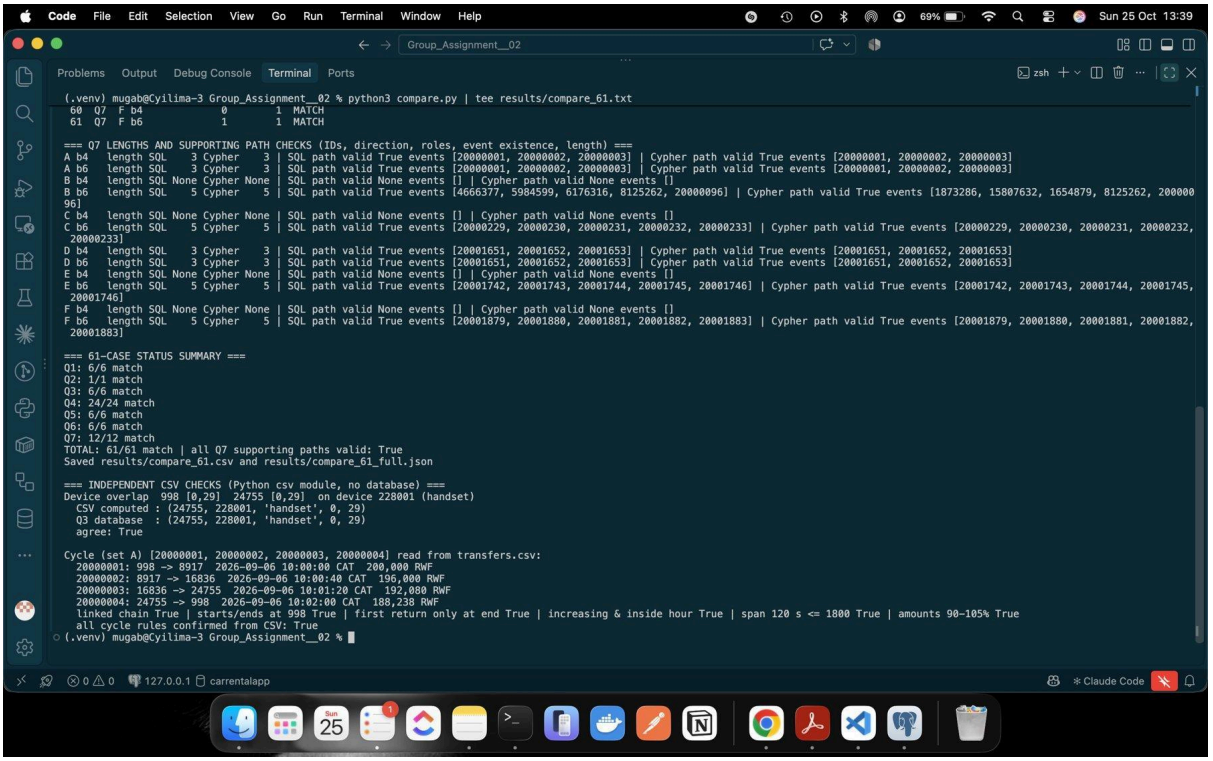


Figure 34: 61/61 summary and the two independent CSV checks

6.2 Timing a fixed selection

Each engine was timed alone with the other service stopped, persistent connections and the same memory settings. Each of the 21 cases had 3 untimed warm-ups (180 s limit) and 5 measured runs (60 s limit), timed with perf_counter from sending the query to receiving the last row. All 210 runs per engine finished, with no timeouts or errors.

Table 14: Median times (min and max in results/timing/comparison_6_2.csv)

Case	PG median ms	Neo4j median ms	PG / Neo4j	Faster
Q1 B / E	0.329 / 0.096	3.540 / 2.537	0.09 / 0.04	PostgreSQL
Q3 B / E	0.890 / 0.137	4.676 / 3.519	0.19 / 0.04	PostgreSQL
Q5 B / E	0.875 / 0.113	3.325 / 2.451	0.26 / 0.05	PostgreSQL
Q6 B / E	0.286 / 0.114	4.567 / 2.424	0.06 / 0.05	PostgreSQL
Q2 month	1,221.0	23,400.7	0.05	PostgreSQL
Q4 B d4 / d6 / d8 / d10	0.711 / 1.345 / 3.113 / 4.886	3.243 / 3.658 / 3.320 / 3.255	0.22 / 0.37 / 0.94 / 1.50	PG until d8, Neo4j d10
Q4 E d4 / d6 / d8 / d10	0.361 / 1.217 / 2.879 / 4.823	2.679 / 3.279 / 3.096 / 3.611	0.13 / 0.37 / 0.93 / 1.34	PG until d8, Neo4j d10
Q7 B b4 / b6	161.6 / 13,815.7	7.574 / 7.497	21 / 1,843	Neo4j
Q7 E b4 / b6	170.7 / 14,301.7	6.054 / 11.443	28 / 1,250	Neo4j

One processing difference (Q2). PostgreSQL used a Parallel Seq Scan with 4 workers over transfers (142,202 pages read), each worker making a partial hash aggregate, joined by Gather Merge: 1,677 ms. Neo4j used one thread to scan all 20,003,300 TRANSFER relationships, read two properties from each (40,006,600 db hits) and build

a time zone datetime for every one (another 40,006,600 db hits): about 100 million db hits and 25.1 s. A table scanned in parallel suits a whole-month report much better.

Q7 plans. The PostgreSQL CTE fwd made 427,955 (account, depth) rows with 60,059,131 buffer hits and wrote temp files, so it explored nearly every person account up to depth 6. Neo4j's ShortestPath used 25,634 db hits and stopped as soon as a shortest path was found.

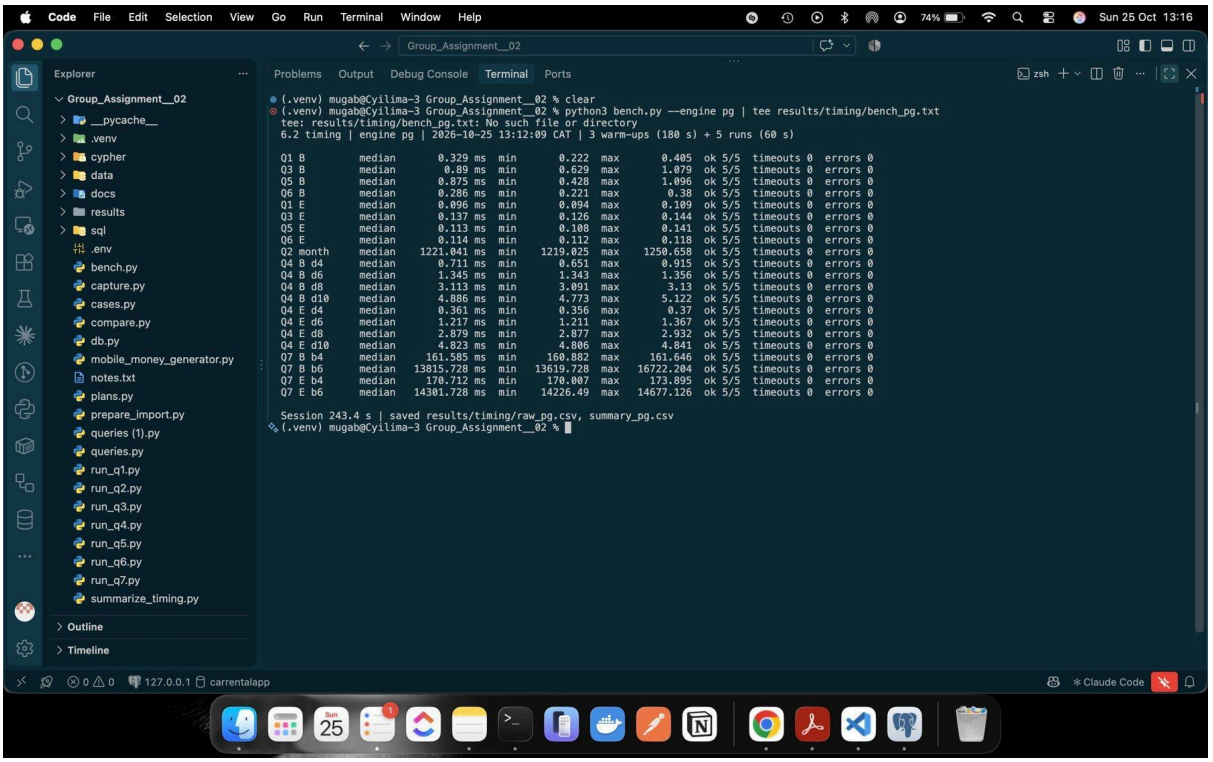


Figure 35: PostgreSQL timing: 21 cases, 5/5 runs each

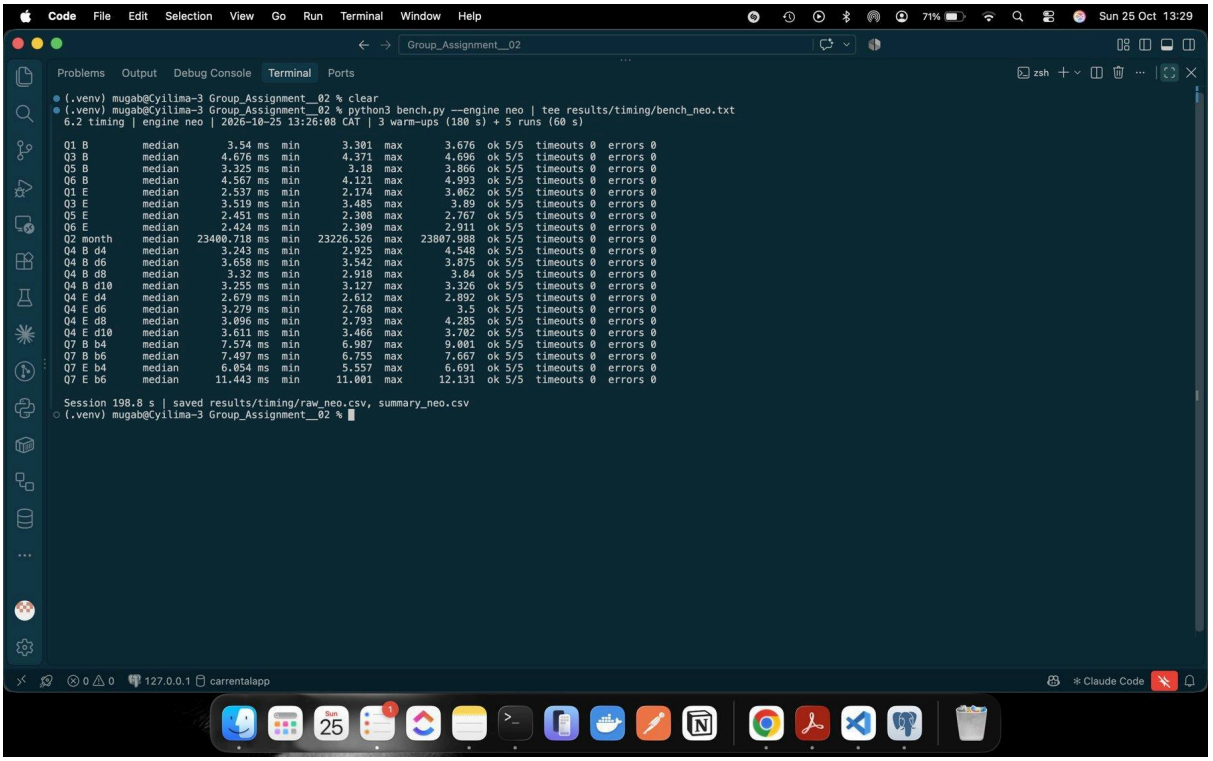


Figure 36: Neo4j timing: 21 cases, 5/5 runs each

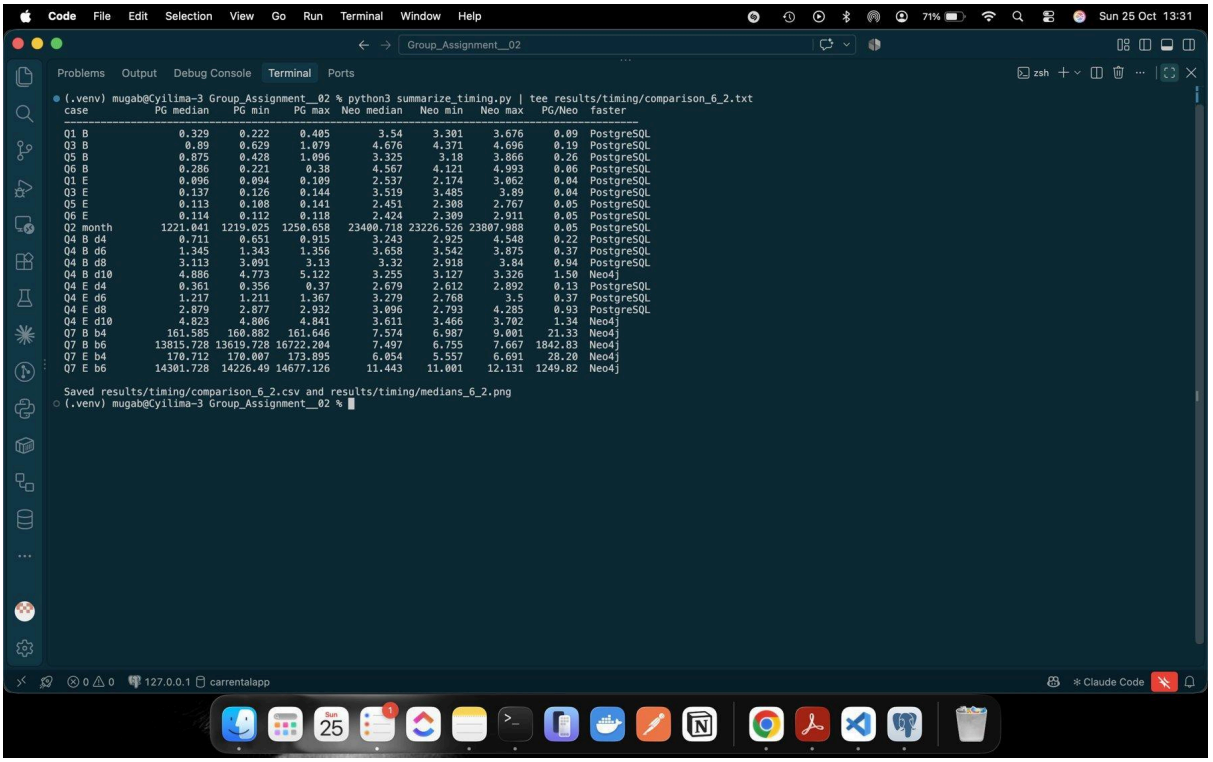


Figure 37: Combined medians and PostgreSQL/Neo4j ratios

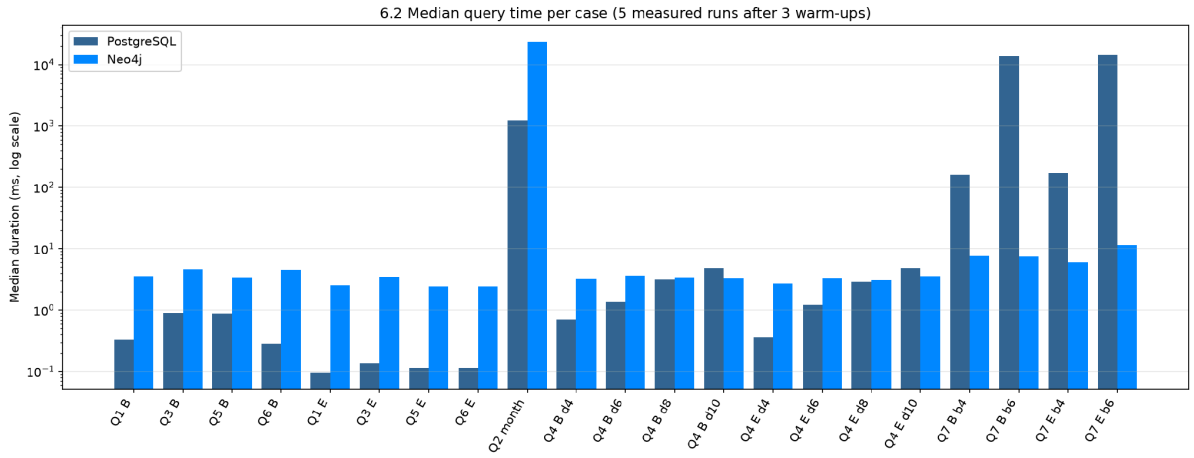


Figure 38: Median query time per case (log scale)

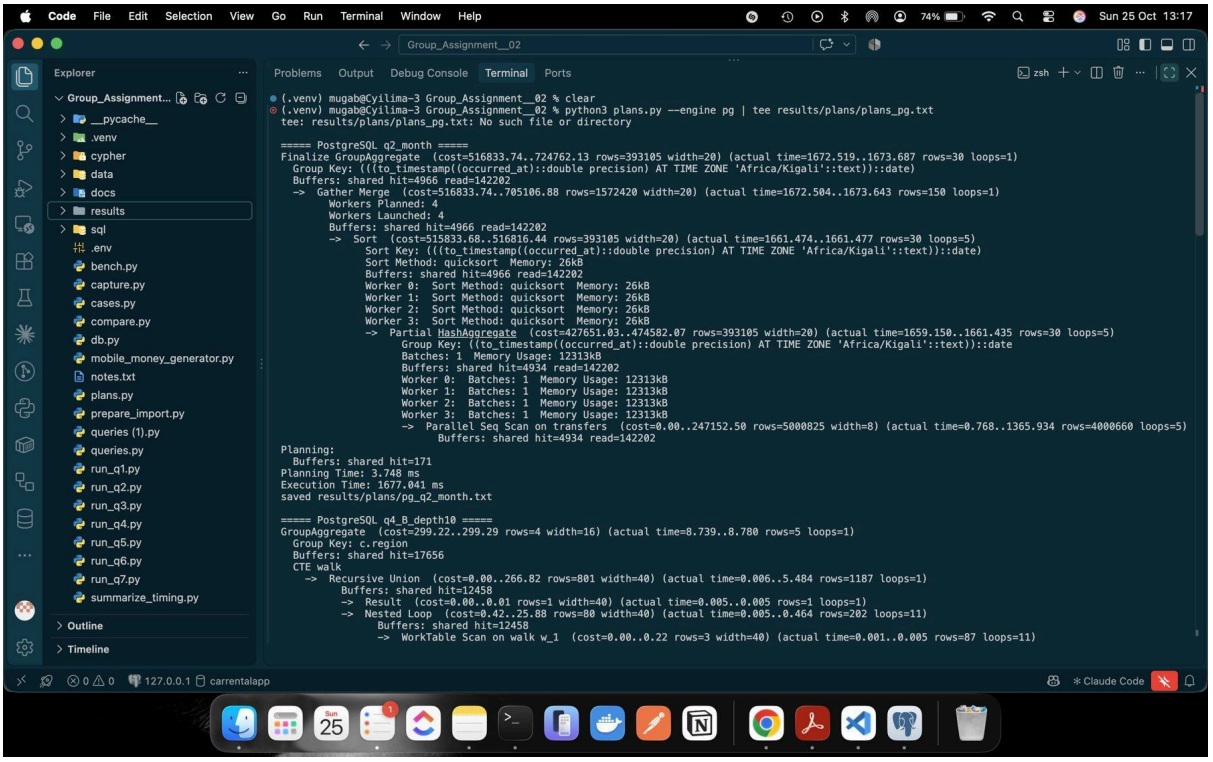


Figure 39: PostgreSQL plans: Q2 parallel scan and start of Q4

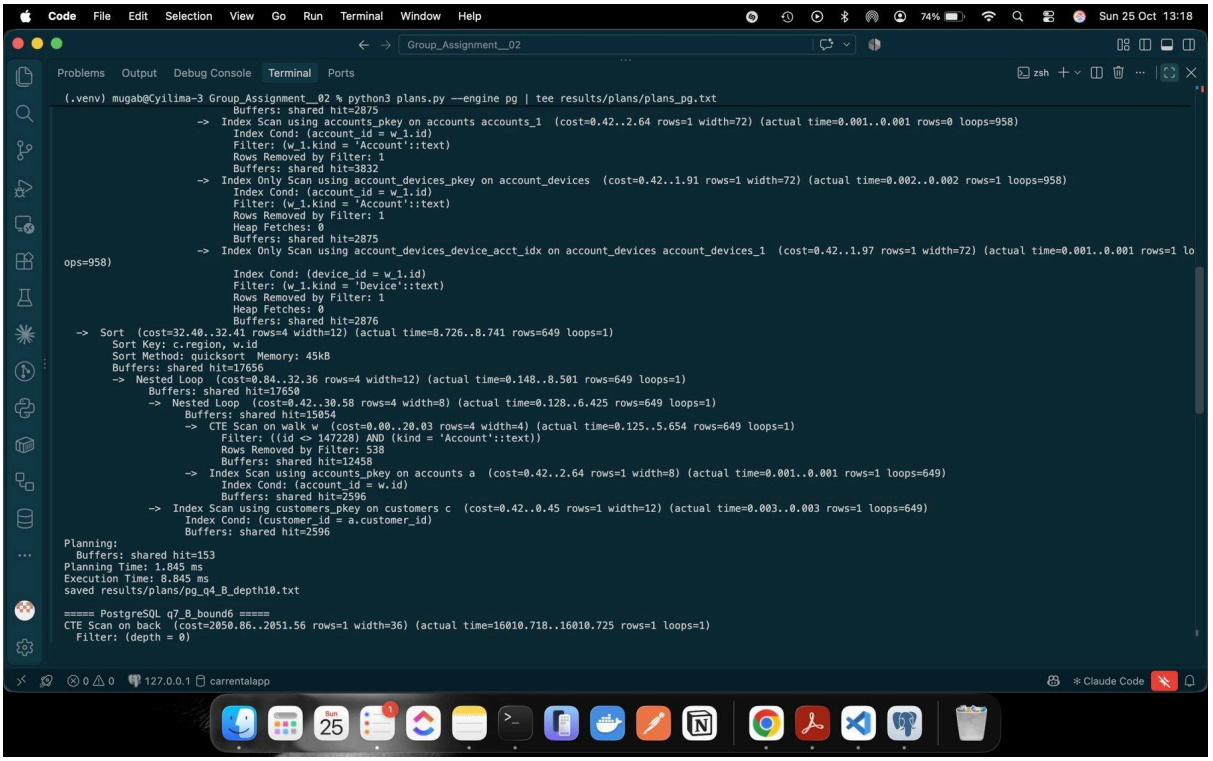


Figure 40: PostgreSQL plan: end of Q4 B depth 10

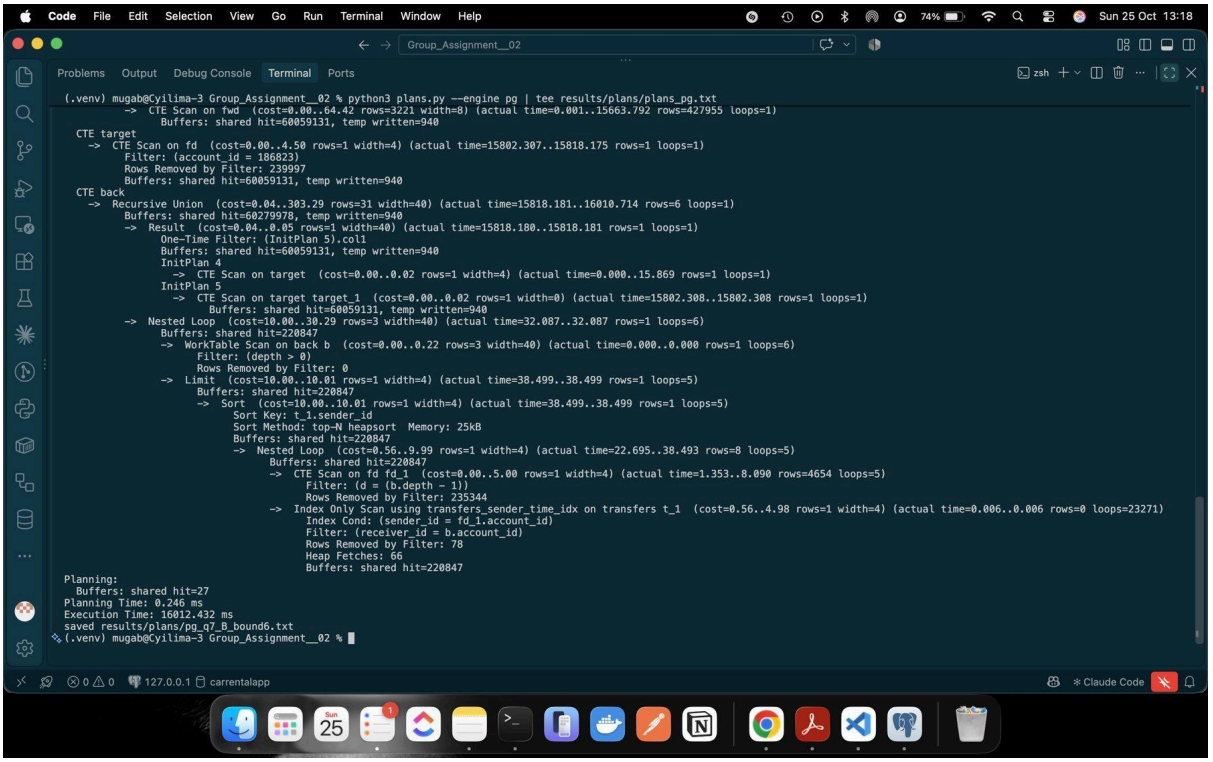


Figure 41: PostgreSQL plan: Q7 B bound 6 (60 million buffer hits)

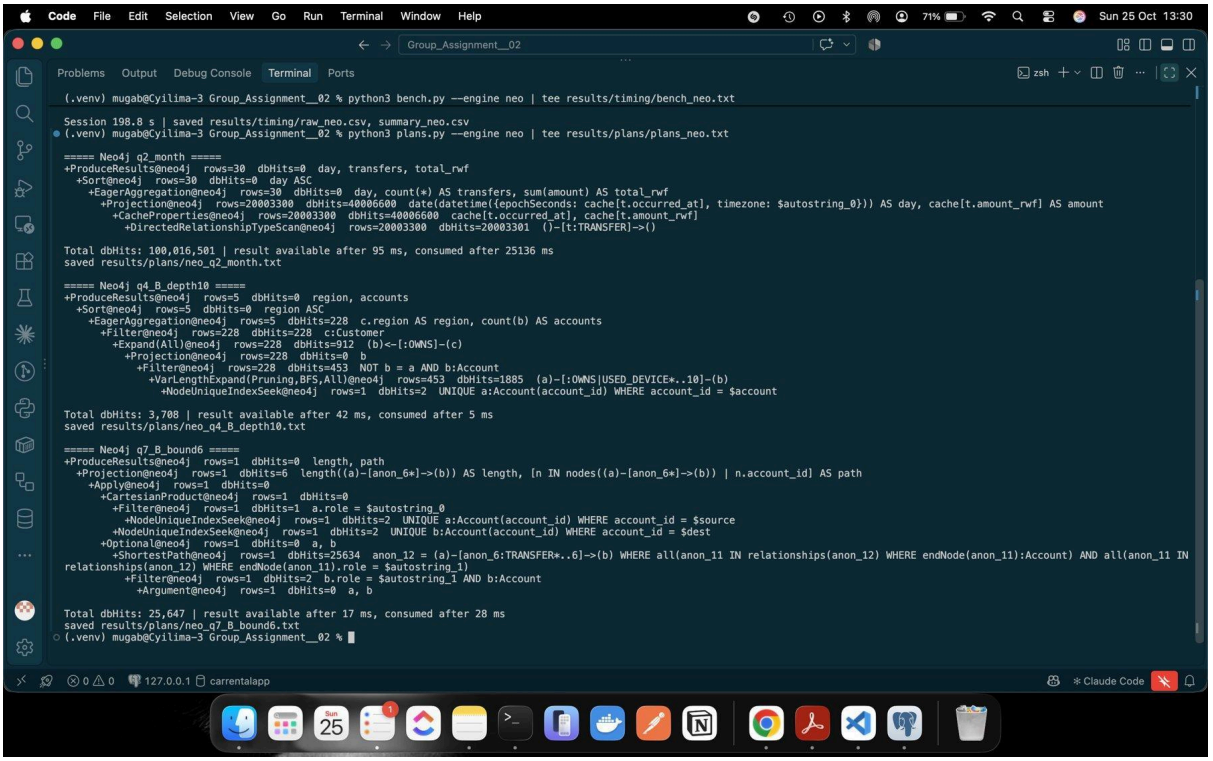


Figure 42: Neo4j PROFILE for Q2, Q4 B depth 10 and Q7 B bound 6

6.3 One SQL improvement (Q7)

Wasted work: the original plan expanded the whole person network to depth 6 from one side only and expanded the same accounts again at every depth. **Change:** the new query (Q7_SQL_V2) searches forward from the source for ⌈bound/2⌉ steps and backward from the destination for ⌊bound/2⌋ steps, then takes the smallest forward + backward distance over the accounts both searches reached, and rebuilds one path from that meeting account. It is still exact: every route within the bound has an account that both searches reach. The original query was kept for comparison.

All 12 lengths still match Cypher and all 12 paths are valid. The new plan for B bound 6 ran in 72.3 ms under EXPLAIN ANALYZE, against 16,012 ms and 60 million buffer hits before.

Table 15: Q7 before and after the rewrite

Case	Original median ms	Revised median ms	Speed-up	Neo4j median ms
Q7 B b4	161.6	5.548	29x	7.574
Q7 B b6	13,815.7	27.918	495x	7.497
Q7 E b4	170.7	2.205	77x	6.054
Q7 E b6	14,301.7	25.951	551x	11.443

We selected the revised version using the bound-6 medians. At bound 6 the gap to Neo4j fell from about 1,250 to 1,843 times to 2.3 to 3.7 times, and at bound 4 PostgreSQL is now faster than Neo4j.

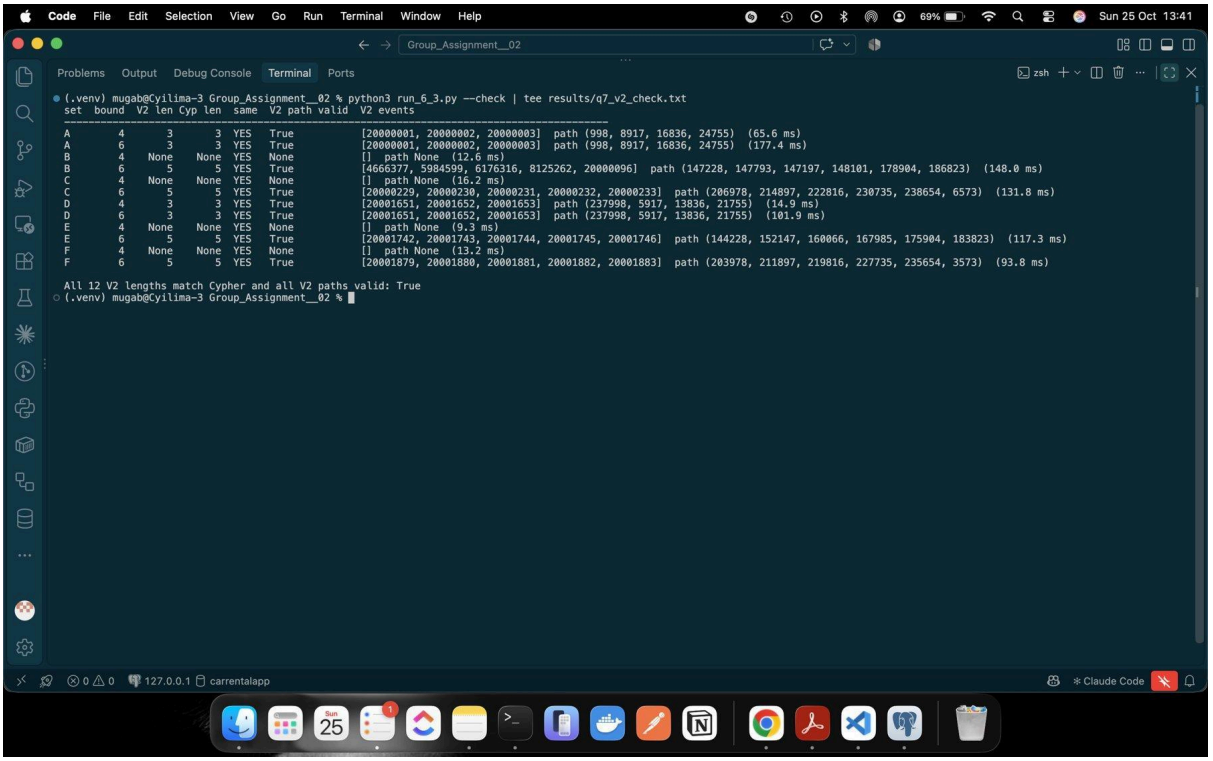


Figure 43: Revised Q7: 12 lengths match Cypher and paths valid

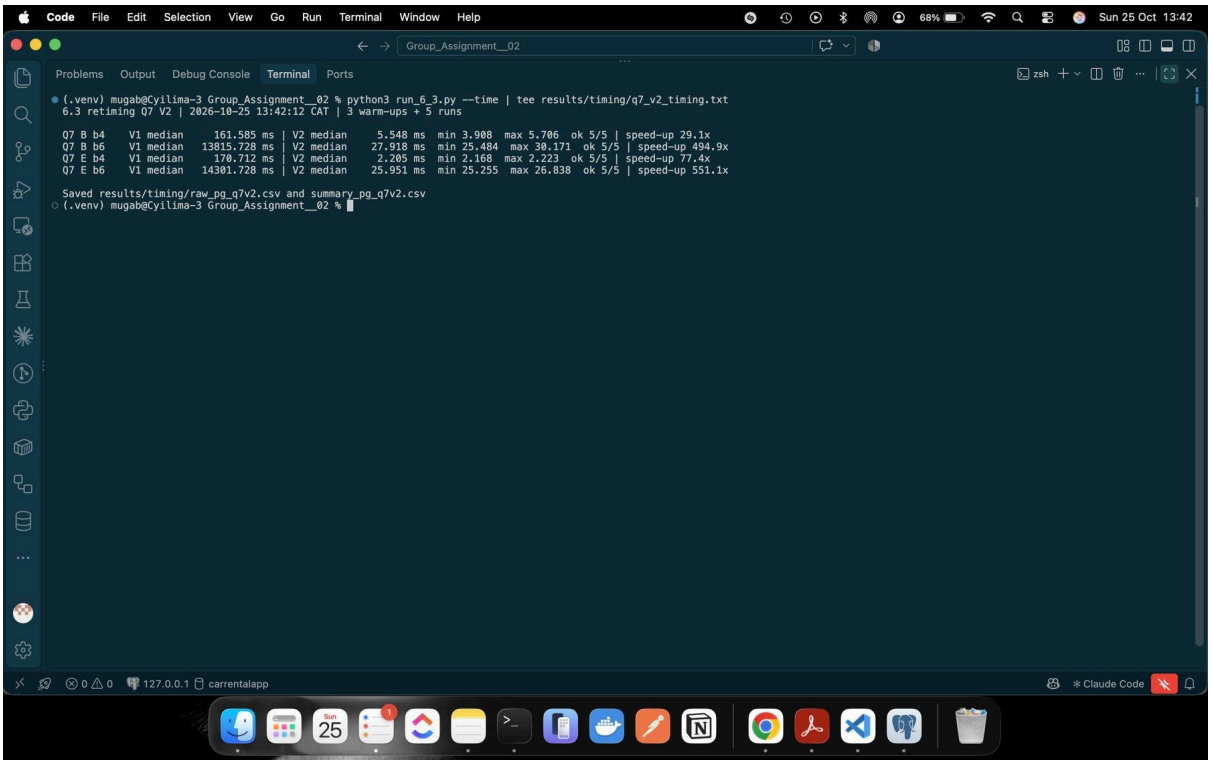


Figure 44: Revised Q7 timing against the original

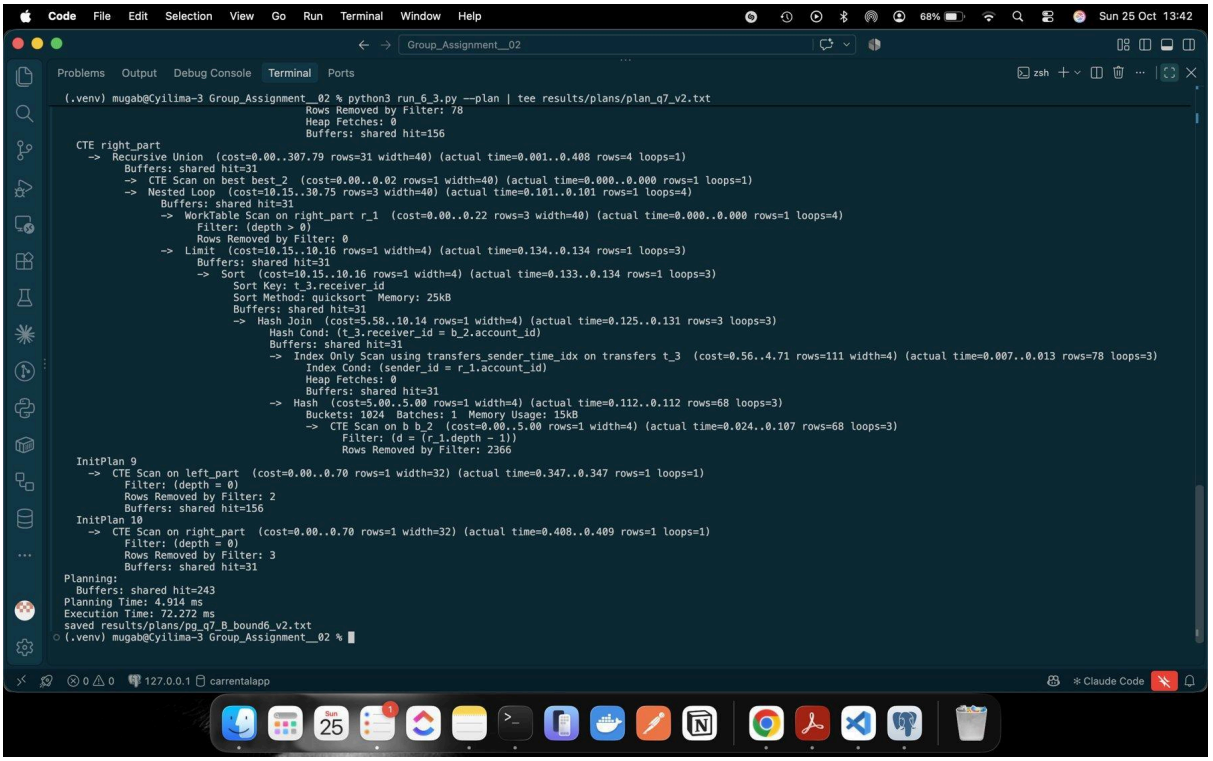


Figure 45: Revised Q7 plan (72.3 ms)

7 Concurrent requests

7.1 Load test

loadtest.py starts one process per client, each with its own persistent connection. Each client warms up A-B-C once and waits at a Barrier; when all are ready one client records the common start time and the 15 s deadline. Clients start cycles until the deadline, finish any started cycle and send the next request straight after the previous one, with the 60 s timeout. Answers are checked after timing against reference answers. Q7 uses the revised length-only SQL. A monitor process counts active server queries and CPU. All 48 tests had every request correct, 0 errors and 0 timeouts.

Table 16: Load test summary (median of 3 repetitions; full table in results/load/summary_7.csv)

Workload	Engine	Thr/s 1 client	Thr/s 16 clients	Peak thr/s (clients)	p95 ms 1 → 16
Q4 d10	PostgreSQL	107.67	428.58	439.93 (8)	19.19 → 79.87
Q4 d10	Neo4j	413.98	3,850.24	3,973.18 (8)	5.95 → 8.48
Q7 len b6	PostgreSQL	37.77	103.72	112.66 (4)	27.70 → 169.22
Q7 len b6	Neo4j	265.20	2,432.90	2,432.90 (16)	6.37 → 17.70

The active-query column shows 1, 4, 8 and 16 active queries for PostgreSQL. For Neo4j it shows up to 13 because its 2 to 4 ms queries are often already finished when the monitor looks.

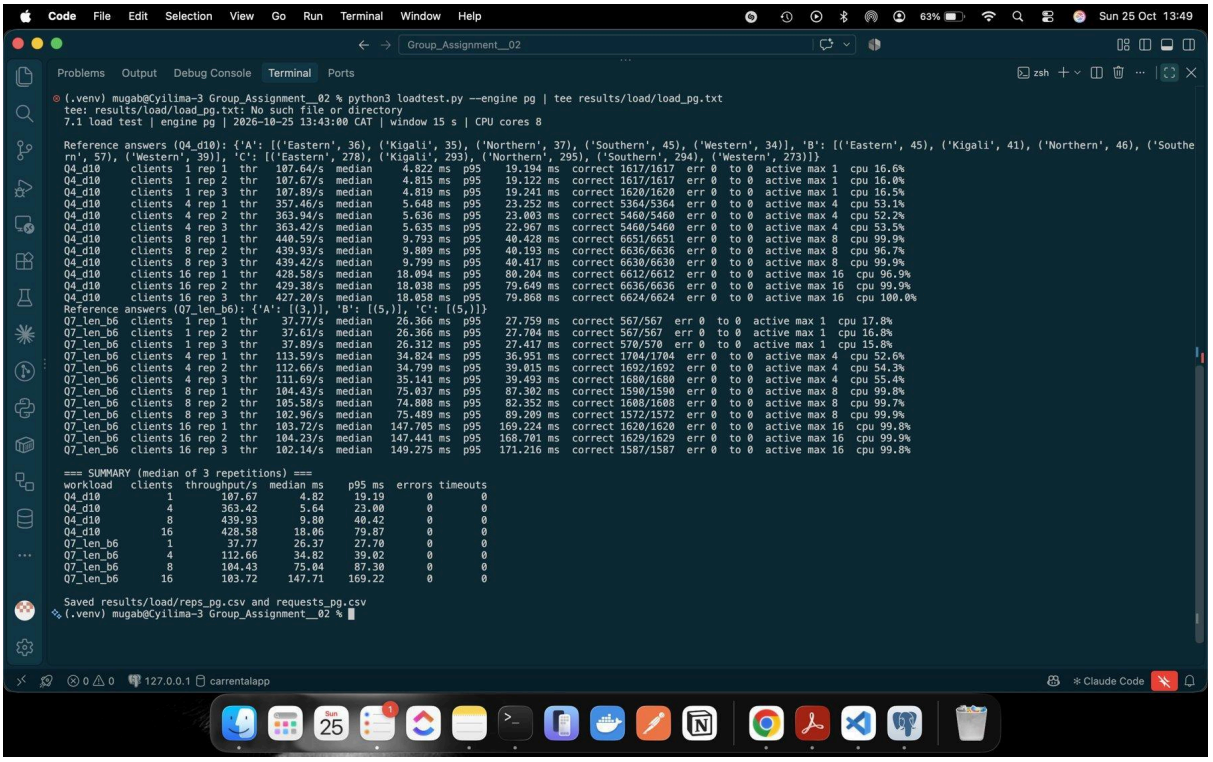


Figure 46: PostgreSQL load test: every repetition, active clients and CPU

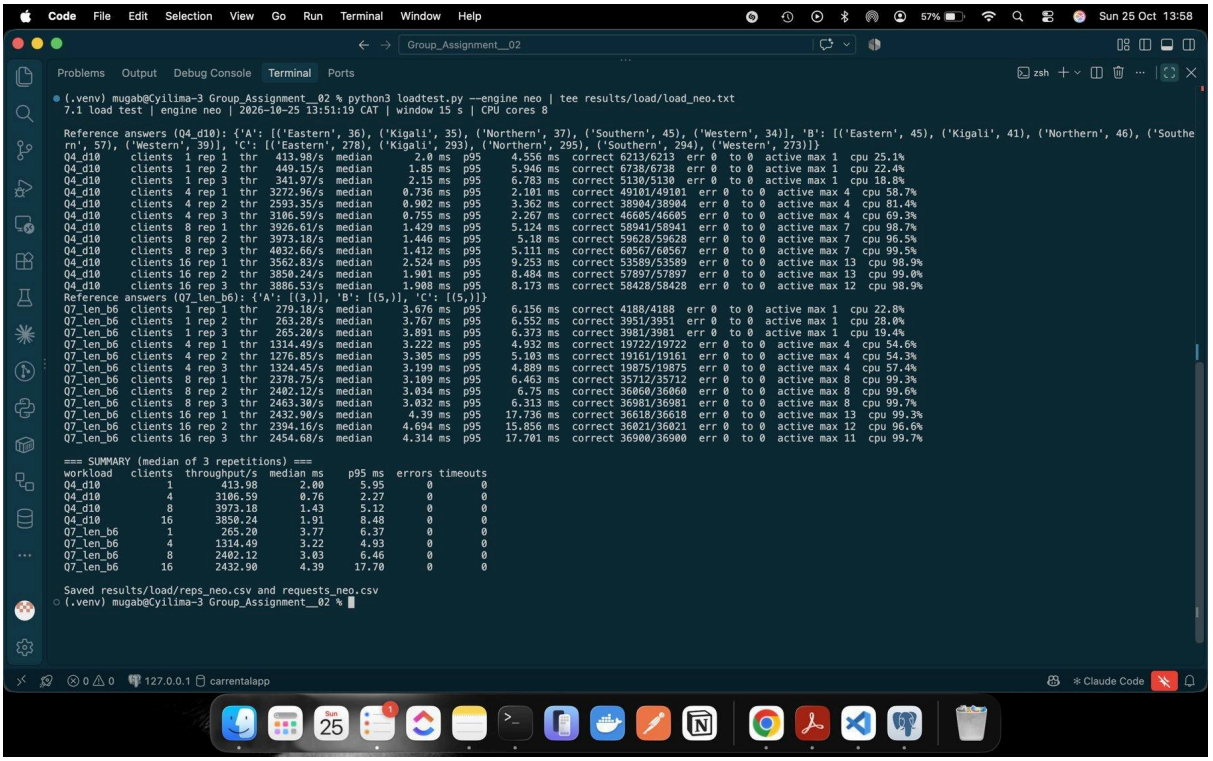


Figure 47: Neo4j load test: every repetition, active clients and CPU

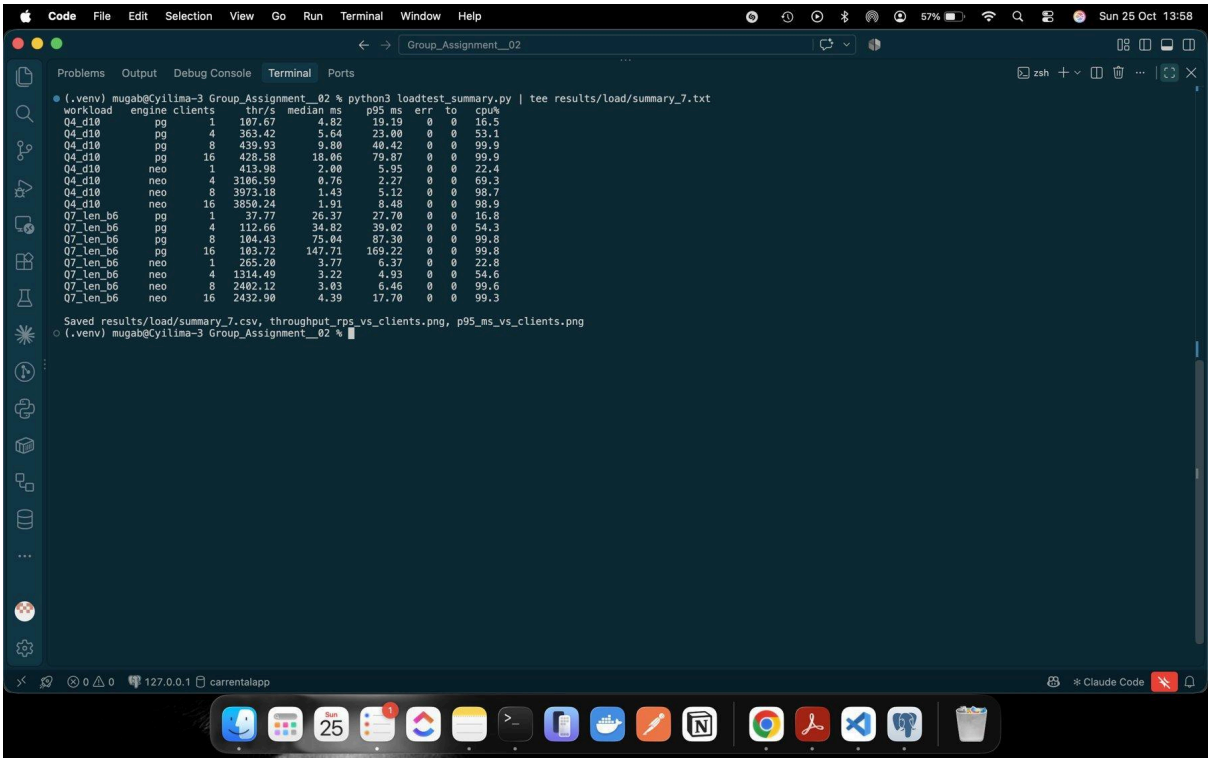


Figure 48: All 16 engine/workload/client combinations

7.2 Scaling

From 1 to 16 clients, PostgreSQL throughput rose 4.0 times for Q4 and 2.7 times for Q7, and Neo4j rose 9.3 and 9.2 times. Both engines **saturate at about 8 clients**: CPU was 97 to 100% at 8 clients on this 8-core M1, and 16 clients add no throughput, only waiting.

PostgreSQL's Q4 median went from 9.80 to 18.06 ms between 8 and 16 clients while throughput stayed at about 430 to 440 per second. The plans explain the different ceilings: both are CPU-bound (the PostgreSQL Q4 plan shows 17,656 buffer hits and no disk reads), but each Neo4j request costs much less (3,708 db hits for Q4 B depth 10, 25,647 for Q7 B bound 6).

Throughput is not response time. At 16 clients PostgreSQL finished 429 Q4 requests per second, but each analyst waited a median 18 ms, 3.7 times longer than when working alone.

Does the graph advantage survive the SQL improvement? In this experiment yes, but it is much smaller. Before the rewrite one Q7 query was about 1,800 times slower in PostgreSQL; with the revised query Neo4j's Q7 throughput is 7.0 times higher at 1 client and 23.5 times at 16, and its Q4 throughput is 3.8 and 9.0 times higher. This only covers our data, these two workloads and one machine where the client processes share the same cores. One result we cannot fully explain: Neo4j's Q4 median at 1 client (2.0 ms) was slower than at 4 clients (0.76 ms), possibly because macOS put a single busy process on an efficiency core, but we did not test this.

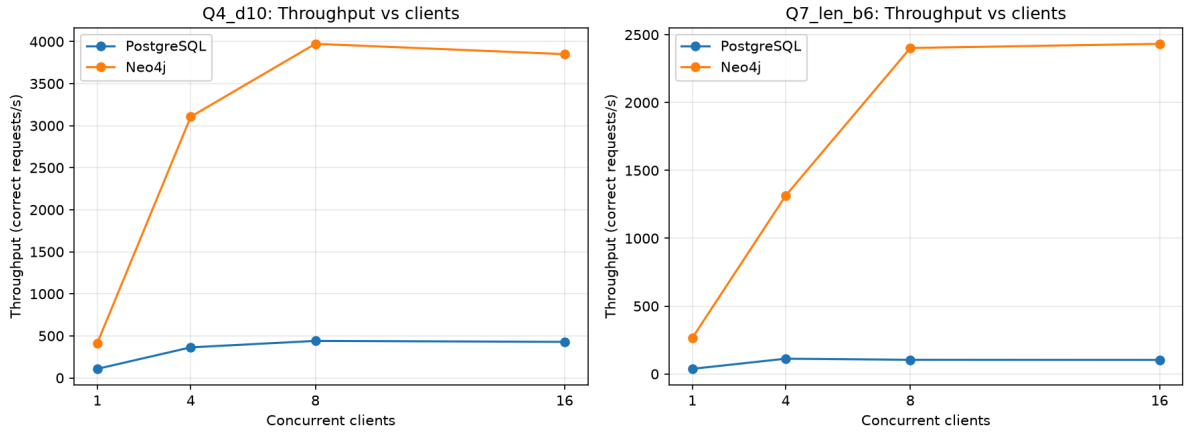


Figure 49: Throughput against clients for both workloads and engines

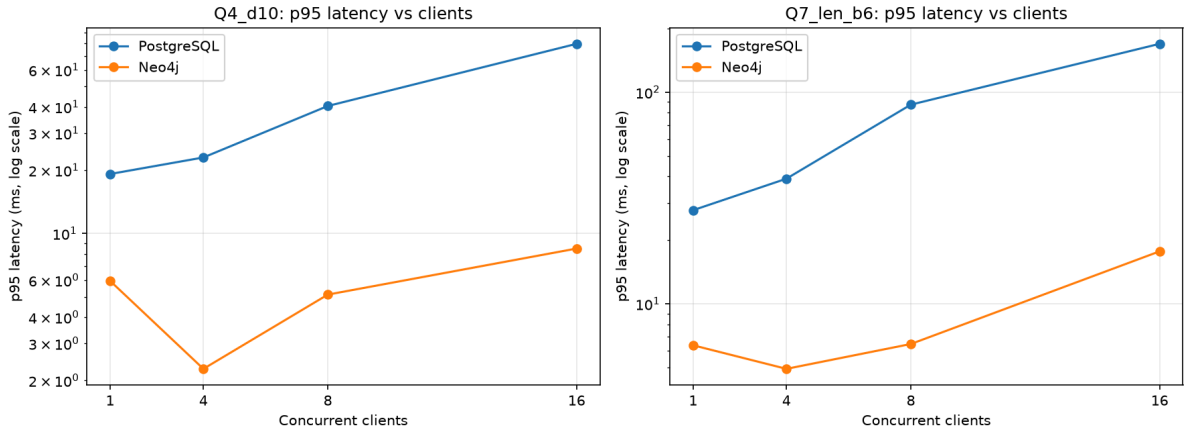


Figure 50: p95 latency against clients (log scale)

8 Decisions

8.1 Prioritise investigation

Table 17: Pair comparison using Q3 to Q7

Pair	Q3	Q4 d4 → d10	Q5	Q6	Q7 b4 / b6	Review first
A / D	Both share a handset with 3 accounts incl. destination	10 → 187 / 10 → 192	A: 1 / D: 0	A: 1 / D: 0	3/3 both	A
B / E	Both share a handset with 5 accounts incl. destination	46 → 228 / 47 → 230	B: 1 / E: 0	0 / 0	none/5 both	B
C / F	Both share a handset with 5 accounts incl. destination	146 → 1,433 / 147 → 1,426	C: 1 / F: 0	C: 1 / F: 0	none/5 both	C

Device sharing, network size and route length are almost the same inside each pair, so structure alone cannot separate them. The difference is timing and amount: A, B and C start quick sequences of large transfers (40 s apart, 98% passed on each time), while D, E and F move small fixed amounts every two days along the same kind of route. Our order is **C first** (chain and a 6-transfer cycle), then **A** (chain and cycle), then **B** (largest amounts, 1.05 million RWF, chain but no cycle), and D, E and F after them because of their device links only.

Pair A/D in detail. On 6 September 998 sent 200,000 RWF to 8917 at 10:00:00 (transfer 20000001), 8917 sent 196,000 RWF to 16836 at 10:00:40 (20000002), 16836 sent 192,080 RWF to 24755 at 10:01:20 (20000003) and 24755 sent 188,238 RWF back to 998 at 10:02:00 (20000004): four transfers in 120 seconds, each 98% of the previous, with 11,762 RWF (5.9%) kept along the loop. Q3 shows that all four accounts were seen on handset 228001 for days 0 to 29. D (237998) shares handset 228301 with 5917, 13836 and its destination 21755 and also reaches it in three transfers, but they were 45,000 RWF each on 6, 8 and 10 September at 10:00, with no reduction and no return.

Ordinary explanation. Four people sharing one phone and passing money round could be an ikimina (rotating savings group) or a family settling contributions, each keeping a small share or paying fees. **Missing evidence:** KYC showing whether the customers are related or live together, which device started each transfer, balances, payment descriptions and whether the loop repeats on other dates. One household with a known savings group would lower the priority; accounts opened together with shared ID documents, or the loop repeating with growing amounts, would raise it. **These records do not prove fraud.**

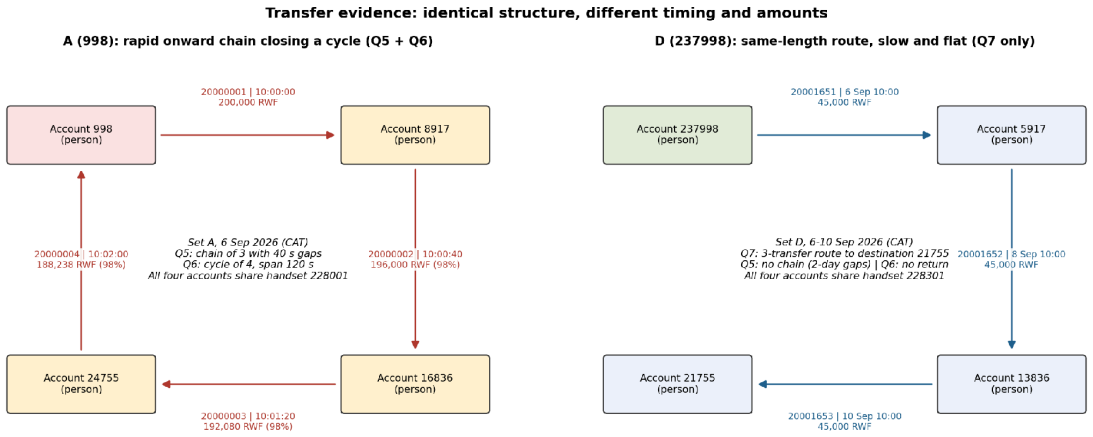


Figure 51: Transfer cycle of A compared with the slow route of D

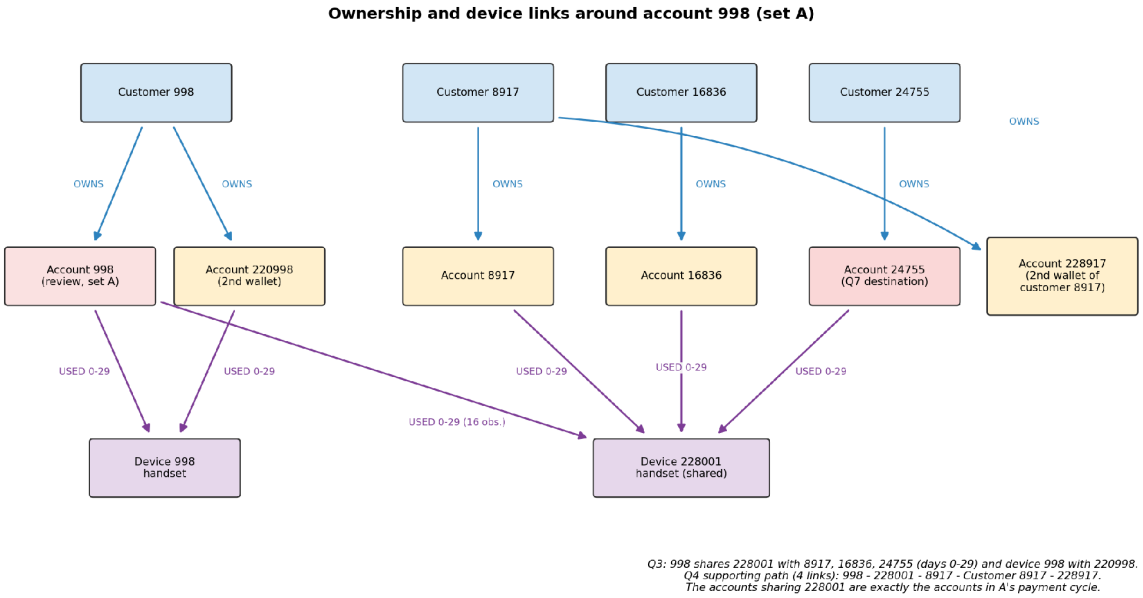


Figure 52: Ownership and device links around account 998

The evidence behind both diagrams is in the Q3, Q5, Q6 and Q7 screenshots in Sections 4.3 and 5.2 to 5.4.

8.2 Recommend a model

Recommendation: PostgreSQL as the system of record and for reporting; Neo4j, fed from PostgreSQL, for multi-step investigation.

Reporting. The monthly report Q2 took 1.22 s in PostgreSQL and 23.4 s in Neo4j, and the indexed lookups Q1, Q3, Q5 and Q6 were 4 to 40 times faster in PostgreSQL. PostgreSQL ran a parallel scan with 4 workers and partial hash aggregation, while Neo4j scanned 20 million relationships on one thread and built a datetime for each (about 100 million db hits). **Traversal.** Neo4j stayed at about 3 ms for Q4 at every depth while PostgreSQL grew from 0.7 to 4.9 ms. For Q7 Neo4j's ShortestPath needed 25,634 db hits (7.5 ms) against 60 million buffer hits (13.8 s) for our first SQL. **Concurrency.** With 16

analysts Neo4j served 3,850 Q4 and 2,433 Q7 requests per second against 429 and 104; both saturated 8 cores at about 8 clients. **Load and storage.** PostgreSQL loaded in 11.0 s plus 44.9 s for keys and indexes and uses 3.72 GB, two thirds of it indexes. Neo4j imported in 22.7 s plus 17.9 s for constraints and uses 2.06 GB.

Evidence against Neo4j. After our rewrite PostgreSQL's shortest path takes 26 to 28 ms, only 2.3 to 3.7 times slower than Neo4j and fast enough for an analyst, and at bound 4 it is faster. Neo4j is also a second system to run, secure and keep in sync.

Model or algorithm? Most of the original Q7 gap was the algorithm: Neo4j stopped as soon as the searches met, the first SQL explored everything to depth 6. Giving SQL a similar strategy shrank the gap about 500 times. The real modelling benefits are that stored relationships make each hop a direct lookup instead of an index probe, and that path patterns express Q5 to Q7 directly; this explains Neo4j's lower cost per request and higher throughput.

Freshness. If transfers are copied into Neo4j, we propose at most 15 minutes of lag with an alert above one hour. Stale data matters because Q5 chains happen within 10 minutes: an investigator could miss a chain in progress, see half a cycle or rely on old device and ownership links, so decisions should be confirmed in PostgreSQL.

Limits. (1) One month of synthetic data on one 16 GB laptop, with the load generator sharing the cores; production volumes and hardware could change the balance. (2) Only seven questions and one SQL improvement were tested; other SQL techniques or Neo4j tuning could move the results either way.

8.3 Individual judgement

Each member's individual judgement is based on a different measured finding and includes the finding, its explanation, and a check that could overturn it.

MUGABO MUTSINZI CHRISTIAN

My individual judgement focuses on the Q7 investigation query and how the SQL implementation affected performance. The original PostgreSQL query took approximately 13.8–14.3 seconds at a path length limit of six. After the SQL query was improved using a bidirectional search approach, the execution time reduced to approximately 26–28 milliseconds. This shows that the performance difference between PostgreSQL and Neo4j was not only related to the database technology but also to how the investigation query was designed.

In my view, this finding is important because it demonstrates that query optimization can significantly improve the performance of relational databases when investigating relationships across multiple accounts. The improved SQL query also produced valid paths, which supports the use of PostgreSQL for this type of investigation when the query is carefully designed.

However, I would not conclude that PostgreSQL will always outperform Neo4j for complex investigations. The result was obtained using one dataset and one improved query. A useful additional check would be to test the optimized SQL and Cypher queries

on larger datasets and different path lengths. This could show whether the performance improvement remains consistent as the investigation becomes more complex.

BUYANGE B. Aline

My individual judgement focuses on the Q2 aggregation query, which was used to calculate daily transaction totals for the CAT investigation. The report shows that PostgreSQL completed this query in approximately 1.22 seconds, while Neo4j took approximately 23.4 seconds. Both databases produced results that reconciled with the captured transaction data.

In my view, this finding supports the use of PostgreSQL for reporting and aggregation tasks. The query required grouping transaction records by day and calculating totals, which is a common requirement in financial investigations. PostgreSQL performed this task faster in the tested environment and provided a suitable approach for producing daily summaries.

However, this result should not be interpreted as proof that PostgreSQL will always be faster for every aggregation. Performance may change depending on dataset size, indexes, query design, and database configuration. One important check would be to repeat the test with a larger dataset and different aggregation periods. We should also confirm that both databases use the same transaction records and aggregation rules. If the results remain consistent, the finding would provide stronger evidence for selecting PostgreSQL for this type of reporting workload.

UWIZEYE CHRISTELLE

My individual judgement focuses on the concurrent load testing results. The report describes 48 load tests and indicates that both database systems completed the tests without errors or timeouts. The results also show that the systems approached saturation at approximately eight concurrent clients. Neo4j achieved higher throughput during the tested load conditions.

In my view, this finding demonstrates that database performance should not be evaluated only by the execution time of individual queries. In an investigation environment, several users may access the system at the same time. Therefore, the ability to handle concurrent requests is also important. The results suggest that Neo4j may be suitable when multiple investigators need to perform relationship-based investigations concurrently.

However, the result does not mean that Neo4j will provide higher throughput in every production environment. The tests were conducted on one laptop, and the load generator shared the available computing resources with the databases. A useful check would be to repeat the tests on dedicated hardware with more memory and processing capacity. Testing additional client levels and different query workloads would also help determine whether the observed saturation point remains the same.